

STUDENT GUIDE

FACULTY OF INFORMATION TECHNOLOGY

PROGRAMMING 632

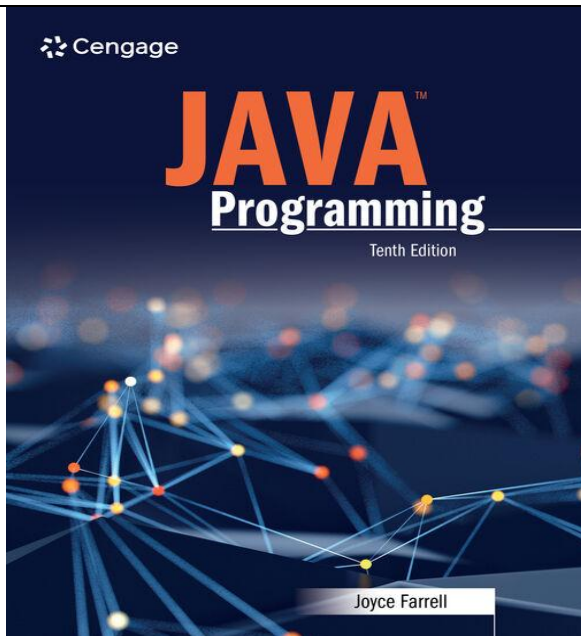
FACULTY OF INFORMATION TECHNOLOGY

LEARNER GUIDE

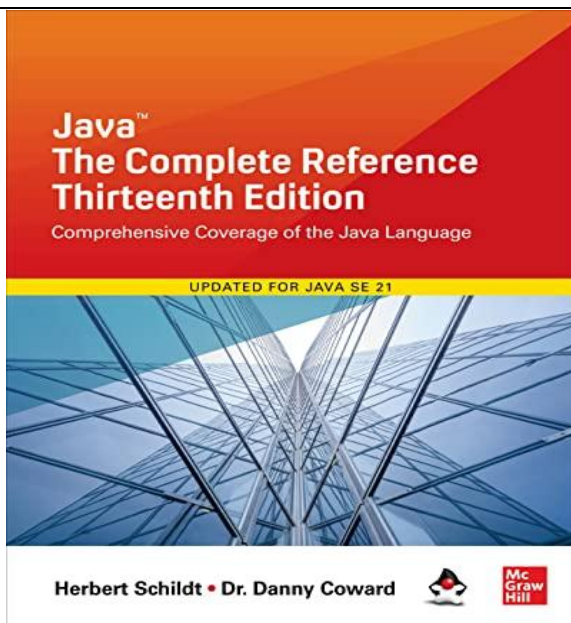
MODULE: PROGRAMMING 632(1ST SEMESTER)

PREPARED ON BEHALF OF

RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY (PTY) LTD



Farrell,J.(2022).Java
Programming.10th
Ed.United
Kingdom: Cengage
Learning.ISBN:
9780357673423



Schildt. H.(2023). Java (The
Complete Reference). 13th Ed.
United Kingdom: Mcgraw-Hill.
ISBN: 9781265058432

Table of Contents

Chapter one: SWINGS AND AWT.....	8
1.1. Features of Swings.....	8
1.1.1 Swing Components	9
1.1.2 Working with Swing	9
1.1.3 Swing Basic Containers.....	10
1.1.4 Panel	10
1.1.5 Windows	10
1.1.6 Frame	10
1.1.7 Canvas	11
1.2. User Interface Components.....	12
1.2.1 JApplet	12
1.2.2 Labels and Icons	12
1.2.3 TextFields	13
1.2.4 Buttons.....	14
1.2.5 JButton Class	15
1.2.6 CheckBoxes	15
1.2.7 Radio Buttons.....	16
1.2.8 Combo Boxes	16
1.2.9 Tabbed Panes	17
1.2.10 Scroll Panes	17
1.2.11 Trees.....	18
1.2.12 Textfields	18
1.2.13 TextArea	19
1.2.14 Layouts and layouts manager.....	20
1.2.15 FlowLayout.....	21
1.2.17 Grid Layout.....	25
1.2.18 Card Layout	25
1.2.19 Timer and Timertask	29

1.2.20 Dialog Boxes	31
1.2.21 Tables	32
1.2.22 Exploring swing	34
1.3 Learner Activity	35
1.4 Coding Activity	35
Chapter Two: NETWORKING IN JAVA ENVIRONMENT	36
2.1 Basics of Networking.....	36
2.1.1 Overview of socket.....	36
2.1.2 Overview of Client / Server	37
2.1.3 Reserved Sockets	37
2.1.4 Proxy server	37
2.2 Inet Address Class	38
2.3 TCP / IP Sockets	41
2.4. Network.....	47
2.5 Learner Activity	47
2.6 Coding Activity	48
Chapter Three: JAVA DATABASE CONNECTIVITY	49
3.1 Features Of The Jdbc API.....	49
3.2 JDBC Interface Levels	51
3.3 The DriverManager Class:	52
3.4 Java.Sql Package	53
Chapter Four: SERVLETS.....	65
4.1 Servlets	66
4.2 Cookies.....	86
4.2.1 Maintaining Session Information	88
4.3 Connecting to Database	90
4.4 Learner Activity	94

The Information Technology (IT) qualification at Richfield College is a dynamic and future-focused program designed to equip students with advanced technical, analytical, and problem-solving skills. At the core of the qualification is a commitment to academic excellence, industry alignment, and innovation, fostering graduates who are proficient in addressing modern technological challenges. This qualification strategically integrates theoretical knowledge with practical applications, preparing students for various roles in the IT sector. The IT program is structured to address the growing complexity of the evolving technological landscape.

The Higher certificate in Information Technology (HCIT) program is a foundational stepping-stone for students who wish to pursue further studies or enter the workforce. Graduates of this program are well-prepared to articulate to the Diploma in IT (DIT) or the Bachelor of Science in IT (BSc IT) qualifications, providing a seamless transition for those seeking to deepen their knowledge and skills in specialized IT areas. Additionally, the program equips students with the essential competencies for entry-level IT roles such as IT Support Technicians, Junior Web/ System Developers, IT Administrators, etc.

The Diploma in Information Technology (DIT) is a comprehensive and practical program designed to build a strong foundation in IT principles while equipping students with the hands-on skills required to meet industry demands. Focused on both theoretical knowledge and applied learning, this qualification prepares students for intermediate-level roles in IT and serves as a stepping-stone for further academic progression or specialization. Graduates of this program are well-prepared to articulate to the Bachelor of Science in IT (BSc IT) qualification. The curriculum covers programming, networking, database management, system analysis etc., ensuring graduates possess the competencies to solve real-world IT challenges effectively.

The Bachelor of Science in IT (BSc IT) program is structured to address the growing complexity of the evolving technological landscape. Through carefully curated modules, students gain a deep understanding of software development, database management, cloud computing, cybersecurity, IT management, artificial intelligence, machine learning, networking etc. Graduates of this program are well-prepared to articulate to the Bachelor of Science Honours in IT qualification. The curriculum is designed to bridge the gap between academic learning and real-world applications, thus fostering innovation and an entrepreneurial mindset. Students are encouraged to participate in research and practical learning.

The programming focus within the IT qualification exemplifies academic innovation and professional alignment. By integrating a diverse range of programming languages with practical application, the curriculum prepares students to excel in the rapidly evolving tech industry. The program aligns with industry

courses from globally recognized leading tech giants, such as Oracle, AWS, IBM, etc ensures that graduates possess the credentials to validate their expertise in software development and cloud-based technologies. This blend of foundational knowledge, practical experience, and industry-standard courses prepares students for immediate employment and establishes a strong basis for long-term career advancement in software development.

Programming 632 expands on the foundational Java programming skills established in Programming 631 by delving into advanced topics essential for developing dynamic and enterprise-level applications. In this module, students master graphical user interface development using Swing and AWT, enabling them to create responsive and visually engaging applications. The curriculum also covers Networking in Java, equipping students with the skills to develop applications that communicate over networks effectively. Additionally, the module introduces Java Database Connectivity (JDBC), providing practical experience in integrating Java applications with databases for seamless data operations. Finally, students explore Servlets, which form the backbone of dynamic, server-side web applications. Through hands-on projects and real-world scenarios, Programming 632 prepares students to build sophisticated, integrated systems that bridge the gap between desktop and web-based solutions.

Chapter one: SWINGS AND AWT



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Describe the features of Swings
- User Interface Components
- Text Fields
- Layouts and Layouts Manager
- Timer and TimerTask
- Exploring Swing

INTRODUCTION

You are already familiar with AWT (Abstract Window Toolkit) classes that help to build user interfaces. Let's look at a glance to its alternative called Swings. A set of classes that provides more powerful and flexible components than AWT are called Swings. Swing provides several citing components like tabbed panes, Scroll panes, tree and tables in addition to the buttons, check boxes, and labels provided by AWT. Swing components are written entirely in JAVA and, therefore, are platform independent. Such components are also called **lightweight components**. There is a substantial number of classes and interfaces in the swing packages. We will provide an overview to just a few in this unit. You have to further explore swings on your own.

1.1. Features of Swings

- Swing provides more powerful and flexible components that are possible with AWT. In addition to the familiar components, such as buttons, check boxes, and

labels, Swing supplies several exciting additions, including tabbed panes, scroll panes, trees and tables.

- Swing components are not implemented by platform-specific code. Instead, they are written entirely in Java and, therefore, are platform independent. The number of classes and interfaces in the swing packages is substantial.

1.1.1 Swing Components

Various swing component classes used in this chapter are as follows:

- (a) Abstract Button: Abstract super class for swing buttons.
- (b) Button Group: Encapsulates a mutually exclusive set of buttons.
- (c) Image Icon: Encapsulates in icon.
- (d) JApplet: The swing version of applet.
- (f) JButton: Class of swing push button.
- (g) JCheckBox: Class of swing check box.
- (h) JComboBox: Encapsulates a combo box. (i) JLabel: The swing version of a label (j) JRadioButton: The swing version of radio button.
- (k) JScrollPane: Encapsulates a scroll bar window. (l) JTabbedPane: Encapsulates a tabbed window.
- (m) JTable: Encapsulates a table-based control. (n) JTextField: The swing version of a text field.
- (o) JTree: Encapsulates a tree-based control.

All swing-related classes are contained in javax.swing and its sub packages like javax.swing.tree.

1.1.2 Working with Swing

As mentioned earlier, swing is a large system, and it has many features that you will want to explore on your own. For example, Swing provides toolbars, tooltips, and progress bars. Also, swing components can provide a pluggable look and feel, which means that it easy to substitute another appearance and behaviour for an element. This can be done dynamically. You may design your own look and feel. Frankly, the swing approach to GUI components might replace the AWT classes sometime in the future, so familiarizing yourself with it now is a good idea.

1.1.3 Swing Basic Containers

The Container class is a subclass of component. It has additional methods that allow other component objects to be nested within it. Other Container objects can be stored inside a container. This makes for a multileveled containment system. A container is responsible for laying out any components that it contains.

1.1.4 Panel

The Panel class is a concrete subclass of Container. It doesn't add any new methods; it simply implements Container. A Panel may be thought of as a recursively nestable, concrete screen component. Panel is the superclass for Applet. When screen output is directed to an applet, it is drawn on the surface of Panel object. In essence, a Panel is a window that does not contain a title bar, menu bar, or border. This is why you don't see these items when an applet is run inside a browser. When you run an applet using an applet viewer, the applet viewer provides the title and border. Other components can be added to a Panel object by its `add( )` method (inherited from Container). Once these components have been added, you can position and resize them manually using the `setLocation( )`, `setSize( )`, or `setBounds( )` methods defined by component.

1.1.5 Windows

The Window class creates top-level windows. A top level window is not contained within any other object; it sits directly on the desktop. Generally, you won't create Window objects directly. Instead, you will use a subclass of Window called Frame, described next.

1.1.6 Frame

Frame encapsulates what is commonly thought of as a .window. It is a subclass of Window and has a title bar, menu bar, borders, and resizing corners. If you create a Frame object from within an applet, it will contain a warning message, such as `.Java Applet Window,` to the user that an applet window has been created. This message warns users that the window they see was started by an applet and not by software running on their computer.

(An applet that could masquerade as a hostbased application could be used to obtained passwords and other sensitive information without the user's knowledge). When a Frame window is created by a program rather than an applet, a normal window is created.

1.1.7 Canvas

Although it is not part of the hierarchy for applet or frame windows, there is one other type of window that you will find valuable: Canvas. Canvas encapsulates a blank window upon which you can draw. Refer to Fig 1.1

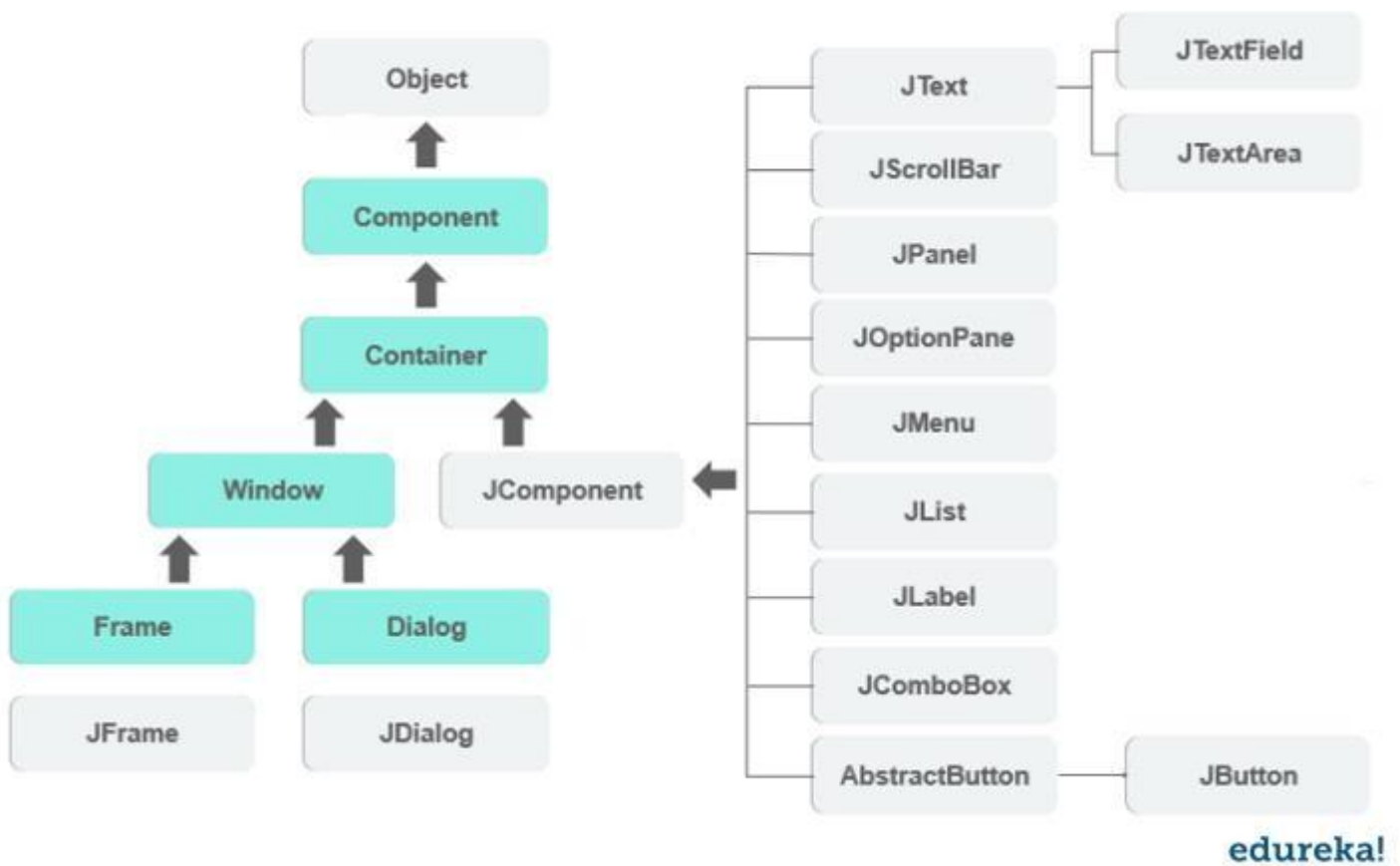


Figure 1.1 - The Class Hierarchy for Panel and Frame
Source:
<https://www.edureka.co/blog/java-swing/>

1.2. User Interface Components

1.2.1 JApplet

JApplet class is the fundamental to the swing. It extends applet class. It is the superclass of applets that uses swings. JApplet supports those functionalities that are not found in Applet. One important difference between Applet and JApplet is that while adding a component to an instance of JApplet, add() method of the Applet should not be invoked. Instead, add() for the content pane of the JApplet object should be called. The content pane can be obtained with the method shown below: Container getContentPane () To add a component to the content pane, the add() method can be used as follows:

```
Void add (comp)
```

Where comp is the component to be added to the content pane.

1.2.2 Labels and Icons

Icons are encapsulated by the Image Icon class in swings, which paints and icon from an Image. Its two constructors are as follows:

```
Image Icon (string filename)  
Image Icon (URL url)
```

The first form of constructor uses the image in the file named filename and the second form of constructor uses the image in the resource identified by url. The Icon interface that declares the methods implemented by the Image Icon class are as follows:

- (a) **int get Icon Height ()** : Returns the icon.s height in pixels.
- (b) **int get Icon Width ()** : Returns the width of the icon in pixels.
- (c) **Void paint Icon (component comp, Graphics g, int x, int y):**

Paints the icon at position x, y on the graphics context g·Comp provides the additional information about the paint operation. Instances of the JLabel class, which extends Jcomponent are the swing labels. It can display text and/or an icon. Some of its constructors are as follows:

```
Jlabel(Icon i)  
Label (String s)
```

Jlabel (sring s, Icon i, int align)

Here s and I are the text and icon used for the label. The align attribute is LEFT, RIGHT, CENTRE, LEADING or TRAILING. Swing constants interface defines these constants. To read icon and label associated with the label, following methods are used:

Icon getIcon()
String getText ()
Void setIcon (Icon i)
Void setText (String s)

Let's consider an example to create and display a label containing both an icon and a string:

```
Import java.awt.*;
Import javax.swing.*;
/* < applet code = "Jlabel Demo" width=250
height=200><lapplet*/
Public class Jlabel Demo extends Japplet
{
    public void init ( )      { // Get Context
pane    container content
Pane=getContentPane ( );
        // Create an icon
        ImageIcon ii = New ImageIcon ("Italy.gif");
        // Create a label
        Jlabel jl = new Jlabel ("Italy, ii,
Jlabel.RIGHT);
        // Add label to the content pane
        contentPane.add (jl);
    }
}
```

1.2.3 TextFields

JtextComponent class, which extends Jcomponent, encapsulats the swing textfield. The functionality common to swing text components is provided by it. Jtextfield is one of its subclasses, which allows you to edit one line of text. Some of its constructors are as follows:

JTextField ()
JTextField (int cols) (Cols is the number of columns)
JText Field (string s,int cols)- s is the string to be presented) **JTextField (sring s)**

An example to create a text field is as follows;

```

import java.awt.*;
import javax.swing.*;
/*<applet code = "JtextFieldDemo" width = 250 height
= 507
</applet> */ public class
JtextFieldDemo extends JApplet
{
    JTextField jtf;
    public void init ( )
    { // get content pane container
        contentPane = getContentPane ( );
        contentPane . setLayout(new
        flowLayout ( ));
        //Add text field to contentPane Jtf
        = new JTextField (15);
        contentPane.add(jtf);    } }

```

1.2.4 Buttons

Some features that are not found in the Button class defined by AWT are provided by the swing buttons.

For example, an icon can be associated with a swing button. Swing buttons are subclasses of the Abstract Button class, which extends JComponent. The methods contained in the Abstract Button Class are as under:

```

Void setDisabledIcon (icon di) Void
setPressedIcon (icon pi)
Void setSelectedIcon (icon si)
Void setRolloverIcon (icon ri)

```

Here **di**, **pi**, **si** and **ri** are the icons to be used for these different conditions. The text associated with a button can be read or written via the following methods:

```

String getText ( )
Void setText (string s)

```

Where **s** is the text to be associated with the button. Concrete subclasses of AbstractButton generate action events when they are pressed. The methods used by listeners to register and unregister for these events are:

```
Void addActionlistener (Actionlistener al)
Void removeActionlistener (Actionlistener al)
```

Where **al** is the actionlistener.

1.2.5 JButton Class

The JButton class provides the functionality of a push button. It allows an icon, a string or both to be associated with the push button. Its constructors are:

```
JButton(Icon i)
JButton(String s)
JButton (String s, Icon i)
```

Where **s** and **i** are the string and icon used for the button.

1.2.6 CheckBoxes

A concrete implementation of AbstractButton is the JCheckBox class. It provides the functionality of a check box. Its constructors are:

```
JCheckBox (Icon i)
JCheckBox (Icon i, Boolean state)
JCheckBox (string s)
JCheckBox (string s, Boolean state) JCheckBox
(string s, icon i)
JCheckBox (string s, Icon i, Boolean state)
```

Where **i** is the icon for the button, **s** specifies the text. If state is true, the text box is initially selected.

Otherwise, it is not. To change the state of the checkbox, the following method is used:

```
Void setselected (Boolean state)
```

Where, state is true if the checkbox should be checked.

1.2.7 Radio Buttons

Radio buttons are supported by the `JRadioButton` class. This is a concrete implementation of `AbstractButton`. Its constructors are as follows:

```
JRadioButton (Icon i)
JRadioButton (Icon i, Boolean state)
JRadioButton (String s)
JRadioButton (String s, Boolean state)
JRadioButton (String s, Icon i)
JRadioButton (String s, Icon i, boolean state)
```

Where `I` is the icon for the button, the text is specified by `s`. If `state` is true, the button is initially selected. Otherwise, it is not. The `ButtonGroup` class is instantiated to create a button group. Its default constructor is involved for this purpose. The method used to add elements to the button group is:

```
Void add(AbstractButton ab)
```

Where `ab` is a reference to the button to be added to the group. Radio button presses generate action events that are handled by `actionPerformed()`. The `getActionCommand()` method gets the text that is associated with a radio button and uses it to set the text field.

1.2.8 Combo Boxes

Swing provides a combo box through the `JComboBox` class, which extends `JComponent`. Normally one entry is displayed by a combobox. However, a drop-down list can also be displayed by a combo box that allows a user to select a different entry. Users can also type his selection into the text field. `JComboBox`'s constructors are:

```
JComboBox ( )
JComboBox (Vector v)
```

Where `v` is a vector that initializes the combo box. Items can be added to the list of choices via the `addItem()` method, whose signature is shown here:

```
Void addItem(Object obj)
```

Where `obj` is the object to be added to the combo box.

1.2.9 Tabbed Panes

A tabbed pane is a component that appears as a group of folders in a file cabinet. Each folder has a title.

When a user selects a folder, its contents become visible. At a time only one of the folders may be selected. Tabbed panes are commonly used for setting configuration options.

JtabbedPane class encapsulates the Tabbed panes which extends JComponent. Its default constructor is used. Method used to define the Tabs is as follows:

```
Void addTab(String str, Component comp)
```

Where str is the title for the tab, and comp is the component that should be added to the tab. Typically, a JPanel or a subclass of it is added. The general procedure to use a tabbed pane in an applet is outlined here:

- (a) Create a JTabbedPane object.
- (b) Call addTab () to add a tab to the pane.
- (c) Repeat step (b) for each tab.
- (d) Add the tabbed pane to the content pane of the applet.

1.2.10 Scroll Panes

A scroll pane is a component that presents a rectangular area in which a component may be viewed. Horizontal and/or vertical scroll bars may be provided if necessary. JScrollPane class implements scroll Panes in swing, which extents JComponent. Its constructors are:

```
JScrollPane (Component comp)  
JScrollPane (int vsb, int hsb)  
JScrollPane (Component comp, int vsb, int hsb)
```

Where comp is the component to be added to the scroll pane, vsb and hsb are int constants that define when vertical and horizontal scroll bars for this scroll pane are shown. These constants are defined by the ScrollPaneConstants interface. The steps that you should follow to use a scroll pane in an applet:

- (a) Create a Jcomponent object.
- (b) Create JScrollPane object.
- (c) Add the scroll pane to the content pane of the applet.

1.2.11 Trees

A tree is a component that presents a hierarchical view of data. A user has the ability to expand or collapse individual subtrees in this display. Trees are implemented in swing by the Jtree class, which extends Jcomponent. Some of its constructors are shown here:

```
JTree (Hashtable ht)
JTree ( Object obj [])
JTree (TreeNode tn)
```

The first form creates a tree in which each element of the hash table ht is a child node. Each element of the array obj is a child node in the second form. The tree node tn is the root of the tree in the third form.

Finally, the last form uses the elements of vector v as child nodes.

1.2.12 Textfields

The Swing text fields is encapsulated by the JtextComponent class, which extends Jcomponent. It provides functionality that is common to Swing text components one of its subclasses is Jtextfield, of its subclasses is JTextField, which allows you to edit oneline of text. Some of its constructors are shown here:

```
JTextField( )
JTextField (int cols)
JTextField (String s, int cols)
JTextField (String s)
```

Here, s is the string to be presented and cols is the number of column in the text field. The following example illustrates how to create a text field. The applet begins by getting its content pane, and then a flow layout is assigned as its layout manager. Next a JTextField object is created and is added to the content pane.

```
Import java.awt.*;
Import javax.swing.*;
/*
<applet codes = "JtextFieldDemo" width = 300 height
```

```

= 50>
</applet> */
public class JtextFieldDemo extends Japplet {
JTextField ( ) ; public void init( ) {
//Get content pane
Container content Pane, Document pane ();
Content pane.setLayout (new FlowLayout ());
//Add text field content
pane jtf = new JTextField
(15); content
pane.add(jtf);

```

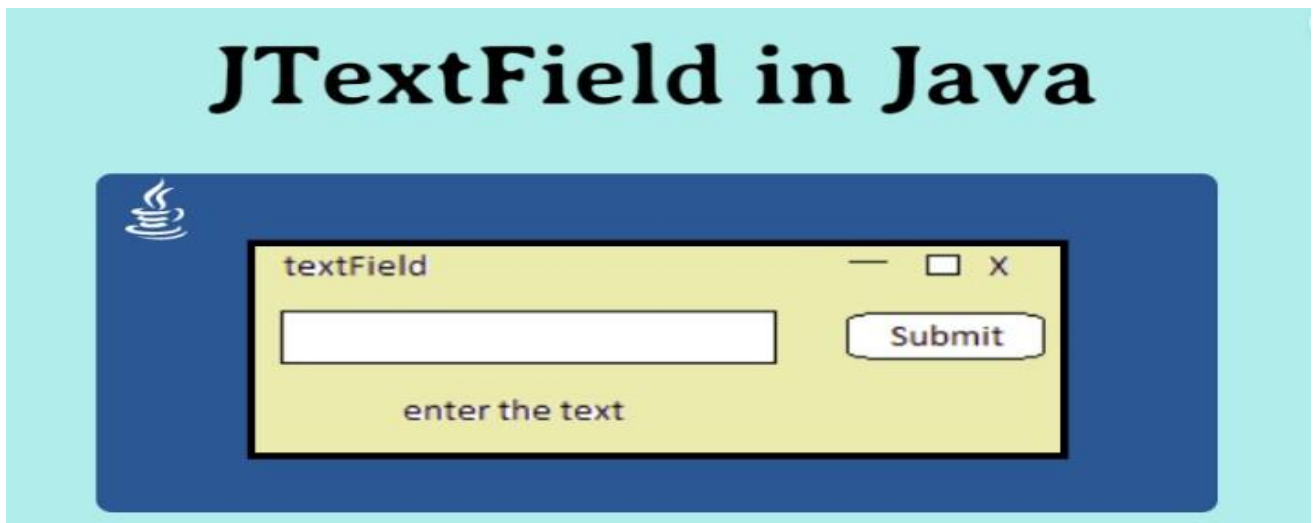


Figure 1.2 - Output from this applet is shown.

Source: <https://www.educba.com/jtextfield-in-java/>

1.2.13 TextArea

Sometimes a single line of text input is not enough for a given task. To handle these situations, the AWT includes a simple multiline editor called TextArea. Following are the constructors for TextArea.

```

TextArea()
TextArea(int numLines, int numChars)
TextArea(String str)
TextArea(String str, int numLines, int numChars)
TexArea(String str, int numLines, int numChars, int sBars)

```

Here, numLines specifies the height, in lines, of the text area, and numChars specifies its width, in characters. Initial text can be specified by str. In the fifth form you can specify the scroll bars that you want the control to have sBars must be one of these values:

```
SCROLLBARS_BOTH SCROLLBARS_NONE  
SCROLLBARS_HORIZONTAL_ONLY SCROLLBARS_VERTICAL_ONLY
```

TextArea is subclass of TextComponent. Therefore, it supports the getText(), set Text(), getSelectedText(), select(), isEditable(), and setEditable() method described in the preceding section. TextArea adds the following methods:

```
Void append(String str)  
Void insert(String str, int index)  
Void replaceRange(String str, int startIndex, int endIndex)
```

The append() method appends the string specified by str to the end of the current text. insert() inserts the string passed in str at the specified index. To replace text, call replaceRange(). It replaces the characters from startIndex to endIndex-1, with the replacement text passed in str.

Text areas are almost self-contained controls. Your program incurs virtually no management overhead. Text areas only generate got-focus and lost-focus events. Normally, your program simply obtains the current text when it is needed.

1.2.14 Layouts and layouts manager

All of the components that we have shown so far have been positioned by the default layout manager. As we mentioned at the beginning of this unit, a layout manager automatically arranges your controls within a window by using some type of algorithm. If you have programmed for other GUI environments, such as Windows, then you are accustomed to laying out your controls by hand. While it is possible to layout Java controls by hand, too, you generally won't want to, for two main reasons. First, it is very tedious to manually lay out a large number of components. Second, sometimes the width and height information is not yet available when you need to arrange some control, because the native toolkit components haven't been realized.

This is chicken and egg situation; it is pretty confusing to figure out when it is okay to use the size of a given component to position it relative to another. Each Container object has a layout manager associated with it. A layout manager is an instance of any class that implements the `LayoutManager` interface. The layout manager is set by the `setLayout( )` method. If no call to `setLayout( )` is made, then the default layout manger is used. Whenever a container is resized (or sized for the first time), the layout manager is used to position each of the components within it. The `setLayout( )` method has the following general form:

`Void setLayout(LayoutManager LayoutObj)`

Here, `layoutObj` is a reference to the desired layout manager. If you wish to disable the layout manager and position components manually, pass null for `layoutObj`. If you do this, you will need to determine the shape and position of each component manually, using the `setBounds( )` method defined by `Component`. Normally, you will want to use a layout manger. Each layout manager keeps track of a list of components that are stored by their names. The layout manager is notified each time you add a component to a container. Whenever the container needs to be resized, the layout manager is consulted via its `minimumLayoutSize( )` and `preferredLayoutSize( )` methods. Each component that is being managed by a layout manager contains the `getPreferredSize( )` and `getMinimumSize( )` methods. These return the preferred and minimum size required to display each component. The Layout manager will honor these requests if at all possible, while maintaining the integrity of the layout policy. You may override these methods for controls that you subclass. Default values are provided otherwise. Java has several predefined `LayoutManager` classes, which are described next. You can use the layout manager that best fits your application.

1.2.15 FlowLayout

`FlowLayout` is the default layout manager. This is the layout manager that the preceding examples have used. `FlowLayout` implements a simple layout style, which is similar to how words flow in text editor. Components are laid out from the upper-left corner, left to right and top to bottom. When no more components fit on a line, the next one appears on the next line. A small space is left between each component, above and below, as well as left and right. Here are the constructors for

```

FlowLayout.
FlowLayout( )
FlowLayout(int how)
FlowLayout(int how, int horz, int vert)

```

The first form creates the default layout, which centers components and leaves five pixels of space between each component. The second form lets you specify how each line is aligned. Valid values for how are follows:

```

FlowLayout.LEFT
FlowLayout.CENTER
FlowLayout.RIGHT

```

These values specify left, center, and right alignment, respectively. The third form allows you to specify the horizontal and vertical space left between components in horz nad vert, respectively. Here is a version of the CheckboxExample shown earlier in this unit, modified so that it used left-aligned flow layout.

```

//Use left-aligned flow Layout()
import java.awt.*;
import java.awt.event.*;
import java.applet.*;

class CheckBoxExample extends Frame
{
    CheckBoxExample()
    {
        setLayout(new
FlowLayout());
        Label lblHobbies = new Label("HOBBIES");
        Checkbox chkSports = new Checkbox("Sports");
        Checkbox chkMusic = new
Checkbox("Music ",true);          Checkbox
chkReading = new Checkbox("Reading");
        add(lblHobbies);           add(chkSports);
        add(chkMusic);             add(chkReading);
    }
}

class
CheckBoxJavaExample
{
    public static void
main(String args[])
    {
        CheckBoxExample frame = new
CheckBoxExample();
    }
}

```

```

frame.setTitle("Checkbox in Java Example");
frame.setSize(350,100);
frame.setResizable(false);
frame.setVisible(true);
    }
}

```

Following is sample output generated by the FlowLayoutDemo applet (Figure 1.3).



Figure 1.3

Source: <https://www.zentut.com/java-swing/java-swing-flowlayout/>

1.2.16 Border Layout

The BorderLayout class implements a common layout style for top-level windows. It has four narrow, fixedwidth components at the edges and one large area in the center. The four sides are referred to as north, south, east, and west. The middle area is called the center. Here are the constructors defined by BorderLayout:

BorderLayout()

BorderLayout(int horz, int vert)

Programming The first form creates a default border layout. The second allows you to specify the horizontal and vertical space left between components in horz and vert, respectively. BorderLayout defines the following constants the specify the regions:

BorderLayout.CENTER BorderLayout.SOUTH

BorderLayout.EAST BorderLayout.WEST

BorderLayout.NORTH

When adding components, you will use these constants with the following form of add(), which is defined by Container.

Void add(Component compObj, Object region);

Here, compObj is the component to be added, and region specifies where the component will be added.

Here is an example of a BorderLayout with a component in each layout area:

```
//Java Program illustrating the
BorderLayout import java.awt.*;
import javax.swing.*; public class
border
{
JFrame
JF;
border()
{
JF=new JFrame();
JButton b1=new JButton("NORTH");
JButton b2=new JButton("SOUTH");
JButton b3=new JButton("EAST");
JButton b4=new JButton("WEST");
JButton b5=new JButton("CENTER");
    JF.add(b1, BorderLayout.NORTH);
    JF.add(b2, BorderLayout.SOUTH);
    JF.add(b3, BorderLayout.EAST);
    JF.add(b4, BorderLayout.WEST);
    JF.add(b5, BorderLayout.CENTER);
    JF.setSize(300, 300);
    JF.setVisible(true);
}
public static void main(String[] args)
{
    new border();
}
}
```

Sample output from the BorderLayoutDemo applet is shown in Figure 1.4.



1.2.17 Grid Layout

GridLayout lays out components in a two dimensional grid. When you instantiate a GridLayout, you define the number of rows and columns. The constructors supported by GridLayout are shown here.

```
GridLayout()  
GridLayout(int numRows, int numColumns)  
GridLayout(int numRows, int numColumns, int horz, int vert)
```

The first form creates a single column grid layout. The second form creates a grid layout with the specified number of rows and columns. The third form allows you to specify the horizontal and vertical space left between components in horz and vert, respectively. Either numRows or numColumns can be zero. Specifying numRows as zero allows for unlimited-length columns. Specifying numColumns as zero allows for unlimited-length.

1.2.18 Card Layout

The CardLayout class is unique among the other layout managers in that it stores several different layouts. Each layout can be thought of as being on a separate index card in a deck that can be shuffled so that any card is on top at a given time. This can be useful for user interfaces with optional components that can be dynamically enabled and disabled upon user input. You can prepare the other layouts and have them hidden, ready to be activated when needed. CardLayout provides these two constructors:

```
CardLayout()  
CardLayout (int horz, int vert)
```

The first form creates a default card layout. The second form allows you to specify the horizontal and vertical space left between components in horz and vert, respectively. Use of a card layout requires a bit more work than the other layouts. The cards are typically held

in an object of type `Panel`. This panel must have `CardLayout` selected as its layout manager. The cards that form the deck are also typically objects of type `Panel`. Thus, you must create a panel that contains the deck and a panel for each card in the deck. Next, you add to the appropriate panel the components that form each card. You then add these panels to the panel for which `CardLayout` is the layout manager. Finally, you add this panel to the main applet panel. Once these steps are complete, you must provide some way for the user to select between cards. One common approach is to include one push button for each card in the deck. When card panels are added to a panel, they are usually given a name. Thus, most of the time, you will use this form of `add()` when adding cards to a panel:

```
Void add(Component panelObj, Object name);
```

Here, `name` is a string that specifies the name of the card whose panel is specified by `panelObj`. After you have created a deck, your program activates a card by calling one of the following methods defined by `CardLayout`:

```
void first (Container  
deck) void last  
(Container deck) void  
next (Container deck)  
void previous  
(Container deck)  
void show (Container deck, String cardName)
```

Here, `deck` is a reference to the container (usually a panel) that holds the cards, and `cardName` is the name of a card. Calling `first()` causes the first card in the deck to be shown. To show the last card, call `last()`. To show the next card, call `next()`. To show the previous card, call `previous()`. Both `next()` and `previous()` automatically cycle back to the top or bottom of the deck, respectively. The `show()` method displays the card whose name is passed in `cardName`.

```
import  
java.awt.*;  
import  
java.awt.event.*  
; import
```

```

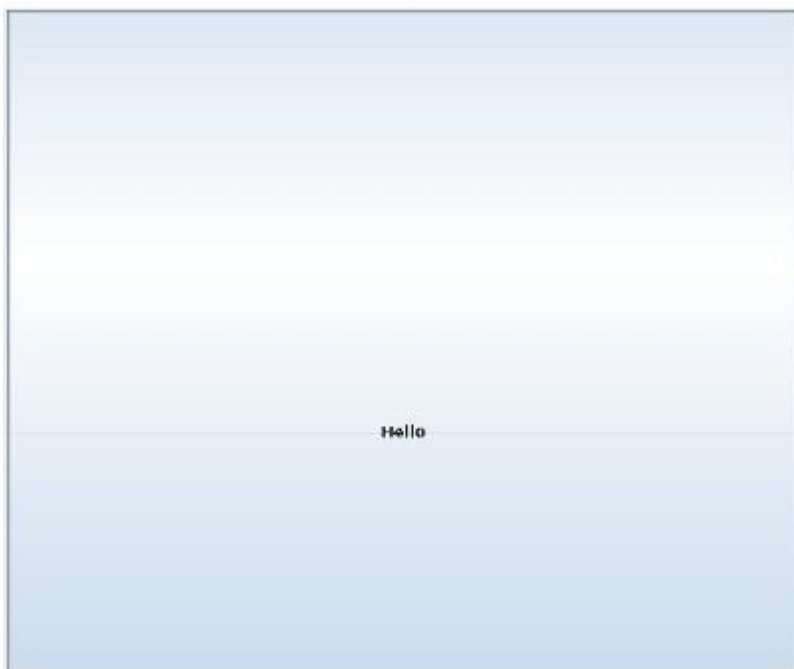
javax.swing.JFrame
me; import
javax.swing.*;
public class Cardlayout extends JFrame implements
ActionListener {
    JButton jb1, jb2, jb3;
    Container con;
    Cardlayout()
    {
        con =
        getContentPane();
        cd = new
        CardLayout(70,
        50);
        con.setLayout(cd);
        jb1 = new
        JButton("Hello");
        jb2 = new
        JButton("Hey");
        jb3 = new
        JButton("Hii");
        jb1.addActionListe
        ner(this);
        jb2.addActionListe
        ner(this);
        jb3.addActionListe
        ner(this);
        con.add("a", jb1);
        con.add("b", jb2);
        con.add("c", jb3);
    }
    public void actionPerformed(ActionEvent e)
    {
        cd.ne
        xt(co
        n);
    }
}

```

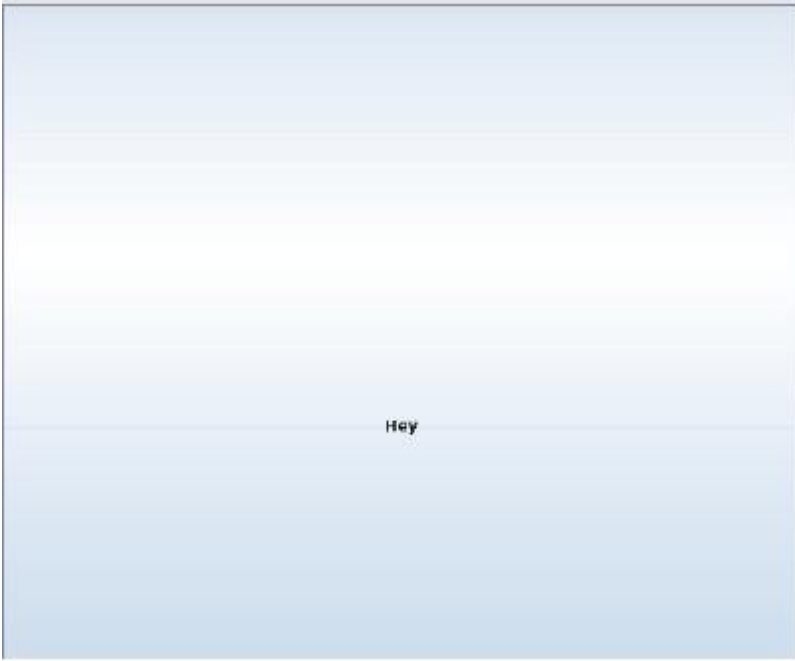
```
}  
public static void main(String[] args)  
{  
    Cardlayout cl1 = new Cardlayout();  
    cl1. setTitle("Checking how Card  
Layout works"); cl1.setSize(800, 800);  
    cl1.setResizable(false);  
    cl1.setVisible(true);  
    cl1.setDefaultCloseOperation(EXIT_ON_CLOSE);  
}  
}
```

Following is the output generated by the CardLayoutDemo applet. Each is activated by pushing is button. You can also cycle through the cards by clicking the mouse (**Figure 1.5**).

So as given below, the output would be a JFrame with the first button with the name “Hello” displayed first:



Clicking on it, the second button, “Hey”, is displayed:



And then, clicking on its button, “Hii” is displayed to the user.

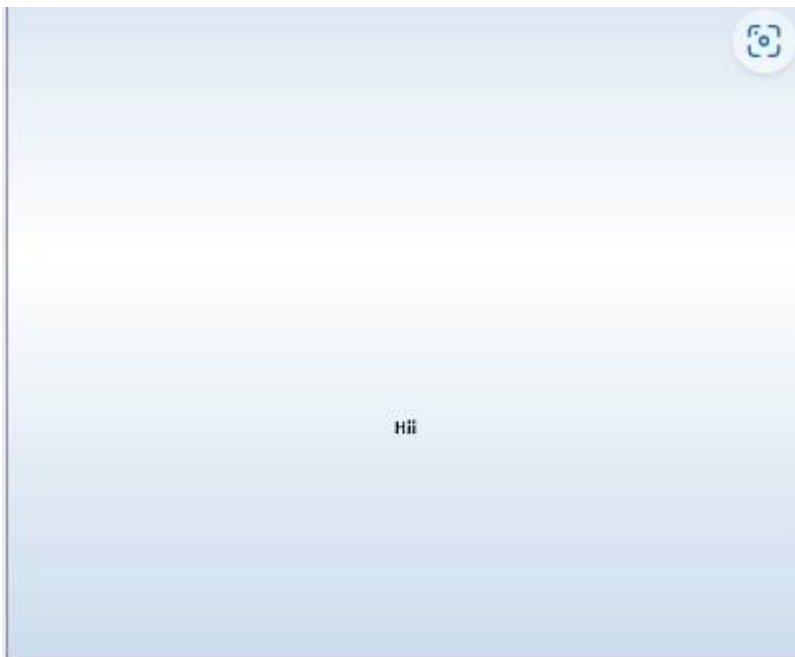


Figure 1.5

1.2.19 Timer and Timertask

Java 2, version 1.3 adds an interesting and useful feature to Java.util: the ability to schedule a task for execution at some future time. The classes that support this are Timer and

TimerTask. Using these classes you can create a thread that runs in the background, waiting for a specific time. When the time arrives, the task linked to that thread is executed. Various options allow you to schedule a task for repeated execution, and to schedule a task to run on a specific date. Although it was always possible to manually create a task that would be executed at a specific time using the Thread class, Timer and TimerTask greatly simplify this process. Timer and TimerTask work together. Timer is the class that you will use to schedule a task for execution. Task being scheduled must be an instance of TimerTask. Thus, to schedule a task, you will first create a TimerTask object and then schedule it for execution using an instance of Timer. TimerTask implements the Runnable interface; thus it can be used to create a thread of execution. Its constructor is shown here:

TimerTask ()

TimerTask defines the methods shown in **Table 9.1**. Notice that run() is abstract, which means that it must be overridden. The run() method, defined by the Runnable interface, contains the code that will be executed. Thus, the easiest way to create a timer task is to extend TimerTask and override run().

Method	Description
Boolean cancel()	Terminates the task. It returns true if an execution of the task is prevented. Otherwise, false is returned.
Abstract void run()	Contains the code for the timer task.
Long scheduledExecutionTime()	Returns the time at which the last execution of the task was scheduled to have occurred.

Table 1.1: The methods Defined by TimerTask

Once a task has been created, it is scheduled for execution by an object of type Timer. The constructors for Timer are shown here.

Timer ()

Timer(boolean DThread)

The first version creates a Timer object that runs as a normal thread. The second uses a daemon thread if DThread is true. A daemon thread will execute only as long as the rest of the program continues to execute.

The methods defined by Timer are shown **Table 1.2**:

Method	Description
Void cancel()	Cancels the timer thread.
Void schedule (TimerTask TTask, long wait)	TTask is scheduled for execution after the period passed in wait has elapsed. The wait parameter is specified in milliseconds.
Void schedule(TimerTask TTask, long wait, long repeat)	TTask is scheduled for execution after the period passed in wait has elapsed. The task is then executed repeatedly at the interval specified by repeat. Both wait and repeat are specified in milliseconds.
Void schedule (TimerTask TTask, Date targetTime)	TTask is scheduled for execution at the time specified by targetTime
Void schedule(TimerTask TTask, Date targetTime, long repeat)	TTask is scheduled for execution at the time specified by targetTime. The task is then executed repeatedly at the interval passed in repeat. The repeat parameter is specified in milliseconds.
Void scheduleAtFixedRate(TimerTask TTask, long wait, long repeat)	TTask is scheduled for execution after the period passed in wait has elapsed. The task is then executed repeatedly at the interval specified by repeat. Both wait and repeat are specified in milliseconds. The time of each repetition is relative to the first execution, not the preceding execution.

Table 1.2: The Methods Defined by Timer

Once a Timer has been created, you will schedule a task by calling `schedule( )` on the Timer that you created. As **Table 1.2** above shows, there are several forms of `schedule( )` which allow you to schedule tasks in a variety of ways. If you create a non-daemon task, then you will want to call `cancel( )` to end the task when your program ends. If you don't do this, then your program may hang for a period of time.

1.2.20 Dialog Boxes

Often, you will want to use a dialog box to hold a set of related controls. Dialog boxes are primarily used to obtain input. They are similar to frame windows, except that dialog boxes are always child windows of a top-level window. Also, dialog boxes don't have menu bars. In other respects, dialog boxes function like frame windows. (You can add controls to them, for example, in the same way that you add controls to a frame window). Dialog boxes may be modal or modeless. When a modal dialog box is active all input is directed to it until it is closed. This means that you cannot access other parts of your program until you have closed the dialog box. When a modeless dialog box is active, input focus can be directed to another window in your program. Thus, other parts of your program remain active and accessible. Dialog boxes are of type `Dialog`. The most commonly used constructors are shown in **Figure 1.6**.

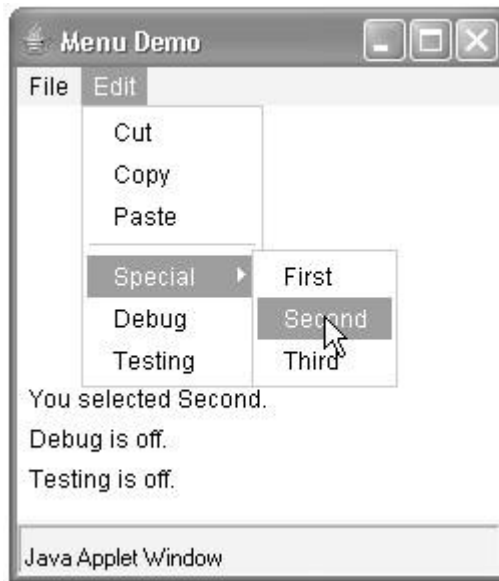


Figure 1.6: Sample output form the MenuDemo Applet

```
Dialog(Frame parentWindow, Boolean mode)
Dialog(Frame parentWindow, String title, Boolean mode)
```

Here, parentWindow is the owner of the dialog box. If mode is true, the dialog box is modal. Otherwise, it is modeless. The title of the dialog box can be passed in title. Generally, you will subclass Dialog, adding the functionality required by your application.

1.2.21 Tables

A table is a component that displays rows and columns of data. You can drag the cursor on column boundaries to resize column. You can also drag a column to a new position. Tables are implemented by the JTable class, which extends Jcomponent. One of its constructors is shown here:

```
JTable(Object data[ ][ ], Object colHeads[ ])
```

Here, data is two-dimensional array information to be presented, and colHeads is a one-dimensional array with the column headings. Here are the steps for using a table in an applet:

1. Create a JTable object.

2. Create a JScrollPane object. (The arguments to the constructor specify the table and the policies for vertical and horizontal scroll bars).
3. Add the table to the scroll pane.
4. Add the scroll pane to the content pane of the applet.

The following example illustrates how to create and use a table: In this example, we will display stock quotes in a table using JTable class.

First we specify the column heading in the columns array.

Then we use two-dimensional array data to store stock quotes data.

Next we create an instance of JTable by passing table data and column heading to the constructor.

Finally we place the table JScrollPane and add it to the main frame.

```
package jtabledemo1;
import
javax.swing
.*; import
java.awt.*;
public
class
Main {
    public static void main(String[]
args) {        final JFrame frame = new
JFrame("JTable Demo");
        String[] columns = {"Code", "Name", "High", "Low",
                            "Close",
                            "Volume","Change","Change %"};

        Object[][] data = {
            {"MBF", "CITYGROUP", 10.16, 10.16, 10.16,
            200, 0.08,0.79},
            {"MBL", "BANK OF AMERICA", 12.66, 12.66, 12.66, 6600,
            0.13,1.04},
            {"MJP", "Morgan Stanley Dean Witter & Co.", 24.97,
            24.97, 24.97, 1000, -0.04,-0.16}

        };

        JTable table = new JTable(data, columns);
        JScrollPane scrollPane = new JScrollPane(table);
        table.setFillsViewportHeight(true);
```

```

JLabel lblHeading = new JLabel("Stock Quotes");
lblHeading.setFont(new
Font("Arial",Font.TRUETYPE_FONT,24));
frame.getContentPane().setLayout(new
BorderLayout());
frame.getContentPane().add(lblHeading,Borde
rLayout.PAGE_START);
frame.getContentPane().add(scrollPane,Borde
rLayout.CENTER);

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
);
frame.setSize(550, 200);
frame.setVisible(true);
    }
}

```

Output from this applet is shown in **Figure 1.7**:



Code	Name	High	Low	Close	Volume	Change	Change %
MBF	CITYGRO...	10.16	10.16	10.16	200	0.08	0.79
MBL	BANK OF ...	12.66	12.66	12.66	6600	0.13	1.04
MJP	Morgan S...	24.97	24.97	24.97	1000	-0.04	-0.16

Figure1.7

Source:<https://www.zentut.com/java-swing/how-to-use-jtable-to-display-data/>

1.2.22 Exploring swing

As mentioned earlier, Swing is a large system, and it has many features that you will want to explore on your own. For example, Swing provides toolbars, tooltips, and progress bars. Also, Swing components can provide a pluggable look and feel, which means that it is easy to substitute another appearance and behaviour for an element. This can be done dynamically. You may even design your own look and feel. Frankly, the Swing approach to GUI components might replace the AWT classes sometime in the future, so familiarizing

yourself with it now is a good idea. Swing is just one part of the Java Foundation Classes (JFC). You may want to explore other JFC features. The Accessibility API can be used to build programs that are usable by people with disabilities.

1.3 Learner Activity

1. What are swings?
2. Describe various features of Swings.
3. Describe various Swing components.
4. Describe basic Swing containers.
5. Define Japplet.
6. Define Tables.

1.4 Coding Activity

Activity: Student Course Selection Form in Java Swing

Objective:

Learn how to create a GUI application using Swing by designing a basic course selection form.

Instructions:

Create a Java class named `CourseSelectionForm` that extends `JFrame`.

Add Swing components, including:

- `JLabel` for student name and course selection.
- `TextField` for student name input.
- `ComboBox` for course selection.
- `Button` for submission. ◦ `TextArea` to display the selected course.

Handle button click events using an `ActionListener`.

Display the selected course when the user clicks "Submit."

Chapter Two: NETWORKING IN JAVA ENVIRONMENT



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Basics of Networking
- Overview of Socket
- Proxy Server
- Domain Naming Service
- Networking Classes and Interfaces
- TCP/IP Sockets
- Datagram Packet

2.1 Basics of Networking

When two or more computers want to communicate with each other to share data or resources then the communications among them can be done through networking.

2.1.1 Overview of socket

A Socket is a handle to a communication link with another application over the network. In other words a socket is a software interface that connects an application to the network. Sockets are often used in client/server applications. A centralized service waits for various remote machines to request specific resources, handling each request as it arrives. In order that clients know how to communicate with the server, standard application protocols are assigned well known ports.

2.1.2 Overview of Client / Server

You might be aware of the term Client/Server mentioned in the context of networking. A client is an entity that relies on another entity to accomplish some task. A server is an entity whose sole purpose is to serve clients by providing them with some kind of well-defined services such as searching a database or accepting mail messages. The interaction between client and server is just like the interaction between a lamp and an electrical socket. The power grid of the house is the server and the lamp is the power client. The server is a permanently available resource whereas the client is free to unplug after it has been served. The notion of a socket allows a single computer to serve many different clients at once, as well as serving many different types of information. This is managed by the introduction of a port, which is a numbered socket on a particular machine. A server process can listen to a port number, although each session is unique. A session process must be multithreaded to manage multiple client connections.

2.1.3 Reserved Sockets

TCP/IP has 65,536 different TCP ports (or sockets) to which every machine can talk.

Table 2.1 defines the standard TCP ports as follows:

Port Name	Port Number	Description
echo	7	Displays whatever is sent by the user
discard	9	Discards whatever is sent by the user.
daytime	13	Gives the local time of the destination machine.
qotd	17	Give the "quote of the day" for the machine.
changer	19	Generates a test stream of characters.
ftp	21	FTP Port.
telnet	23	Telnet Protocol Port.
smtp	25	SMTP Port.
finger	79	Finger Protocol Port.
http	80	Web Server Port.
pop3	110	Pop Version 3 Port.
nntp	119	NNTP Port.

Table 2.1

2.1.4 Proxy server

A proxy server speaks the client side of a protocol to another server. This is often required when clients have certain restrictions on which servers they can connect to. Thus a client

would connect to a proxy server which did not have such restrictions and the proxy server would in turn communicate for the client.

A proxy server has the additional ability to filter certain requests or cache the results of those requests for future use. A caching proxy HTTP server can help reduce the bandwidth demands on a local network's connection to the Internet.

2.1.5 Domain naming services

The Internet wouldn't be a very friendly place to navigate if everyone had to refer to their addresses as numbers. Just as the four numbers of an IP address describe a network hierarchy from left to right, the name of an Internet address, called its domain name, describes a machine's location in a name space from right to left. For example, `www.yahoo.com` is in the `com` domain (commercial sites), it is called Yahoo (company name) and `www` is the name of the specific computer. `www` corresponds to rightmost number in the equivalent IP address.

2.1.6 Networking classes and interfaces

The classes contained in the `java.net` package are listed here

The classes contained in the `java.net` package are listed here

<code>Authenticator (java2)</code>	<code>JavaURLConnection (java2)</code>	
<code>ContentHandler</code>	<code>MultiCastSocket</code>	<code>URL</code>
<code>DatagramPacket</code>	<code>NetPermission</code>	<code>URLConnectionLoader</code>
<code>DatagramSocket</code>	<code>PasswordAuthentication</code>	<code>URLConnection</code>
<code>DatagramSocketImpl</code>	<code>ServerSocket</code>	<code>URLDecoder</code>
<code>HttpURLConnection</code>	<code>Socket</code>	<code>URLEncoder</code>
<code>InetAddress</code>	<code>SocketImpl</code>	<code>ContentHandlerFactory</code>
	<code>SocketPermission</code>	
<code>SocketImplFactory</code>	<code>URLStreamHandlerFactory</code>	<code>FileNameMap</code>
<code>SocketOptions</code>		

2.2 Inet Address Class

As you know, when you want to establish a connection between two machines across the Internet or any other kind of network, addresses are fundamental. The `InetAddress` class is used to convert the Domain Name (or textual Internet address) of an Internet address into an object which represents the address. The `InetAddress` class doesn't have any constructors but it has three factory methods that can be used to create instances of the

InetAddress class. It is noted that according to Object Oriented Program, the term factory applies to a class that can construct instances of that class without invoking constructors. The instance of the class is constructed through static methods built into the class.

2.2.1 Methods (Factory /Instance) of InetAddress Class

Address Class The following are the factory methods of InetAddress Class.

```
static InetAddress getLocalHost( ) throws UnknownHostException. static
InetAddress getByName (String hostname) throws UnknownHostException static
InetAddress [ ] getAllByName (String hostname) throws UnknownHostException
```

The **getLocalHost()** method returns the InetAddress object that represents the local host. The **getByName (String hostname)** method takes the hostname as a parameter and returns the InetAddress object. The **getAllByName (String hostname)** method returns an array of InetAddress objects that includes all the addresses that a particular name (passed as a parameter to the method) resolves to.

Program 1:

```
import java.net.*;
class uptecdemoInetAddress1
{
    public static void main (String args[ ]) throws
UnknownHostException
    {
        InetAddress idr;
        idr = InetAddress.getLocalHost( );
        System.out.println (idr);
    }
}
```

The output produced by the program will be different from the one given below :
tr 7comph/132 . 147 . 168 . 17

Program 2:

The following program displays the name and address of the machine whose host name is given by the user.

```
import java.io.*; import
java.net.*; class
DemoInetAddress2 { public
static void main (String
args[
    ]) throws
UnknownHostException,
```

```

IOException {
    InetAddress adr;
    String host;
    BufferedReader str = new BufferedReader
        (new InputStreamReader (System.in));
    System.out.print ("Enter the Machines |
hostname:");    host = str.readLine( );
    adr = InetAddress.getByName (host);
    System.out.println (adr);
}

```

BufferedReader and InputStreamReader are defined in the java.io package. The BufferedReader class includes methods which allow you to read text lines from a character InputStream. Since the standard input (System.in) is a byte stream, the Input StreamReader class converts the byte stream of the standard input, System.in, into a character stream. The method readLine() allows you to read text lines from an input stream. The output produced by this program is as follows (the output will vary according to the input):

Enter the Machine.s host name : Machine1 Machine1/130. 130. 15. 24 Enter the Machine.s host name : Uptec Uptec / 130. 130. 15. 25

InetAddress class also defines some non static methods which can be used on objects returned by the factory methods (**Table 2.2**).

Methods	Purpose
String getHostName() associated	Returns the string representation of the host name with the InetAddressObject.
byte[] getAddress()	Returns an array of four bytes representing the numeric address of the host. This array of bytes cannot be printed directly by caching them to int. They have to be copied into an array of integers while ANDing with 255 to undo sign-extending.
String toString()	Returns a string that consists of the host name and the IP Address, For example Uptec/130.130.15.25. This function can be overridden.
boolean equals(InetAddress addr)	Returns true if the invoking object has the same Internet address as addr.

Table 2.2

2.2.2 Methods Purpose

String **getHostName()** Returns the string representation of the host name associated with the InetAddressObject. byte[] **getAddress()** Returns an array of four bytes representing the numeric address of the host. This array of bytes cannot be printed directly by caching them to int. They have to be copied into an array of integers while ANDing with 255 to undo sign-extending. String **toString()** Returns a string that consists of the host name and the IP

Address, For example Uptec/130.130.15.25. This function can be overridden. `boolean equals(InetAddress addr)` Returns true if the invoking object has the same Internet address as `addr`. Let us see an example which demonstrates the usage of the `getAddress()` method and overriding of the `toString()` method to display the Internet address.

Program 3:

```
import java.net.*; class
Uptec
{
    static InetAddress adr;
    byte[ ] numadr;    byte[ ]
numadr;    String Addrmsg;
    Uptec( ) throws
UnknownHostException
    { adr =
        InetAddress.getLocalHost(
        ); numadr =
        adr.getAddress( );
        Addrmsg =
        adr.getHostName( ) +"/";
    }    public String
toString( )
    { for (int i = 0; i < numadr.length; i++)
        Addrmsg + = (Numadr[i] & 255) + "."; return
        "The local machine|'s address is : "
        +Addrmsg;
    }
}
```

2.3 TCP / IP Sockets

TCP/IP sockets are used to implement reliable, bidirectional, persistent, point to point, stream based between two hosts on the Internet. A socket can be used to connect Java's Input/Output system to other programs that may reside either on the local machine or any other machine on the Internet.

In Java, TCP sockets are classified as:

- TCP Server Socket
- TCP Client Socket

The java.net package provides two classes that allow you to create the two kinds of sockets. The classes are Socket and Server Socket respectively. The Socket class, basically used to create client sockets, is designed to connect to server sockets and initiate protocol exchanges. The Server Socket class, basically used to create server sockets, is designed to be a listener, which waits for clients to Connect before doing anything. When you create a Socket object, it implicitly establishes the connection between the client and the server. The Socket class provides some constructors to create client sockets

Constructor	Description
Socket (String hostname, int port) throws UnknownHostException, IOException	This constructor is used to create a client socket and connect the local host to the specified hostName and port. It can throw an UnknownHostException or an IOException.
Socket (InetAddress ipAddress, int port) throws IOException	This constructor is used to create a client socket using a pre existing InetAddress object and a port. It can throw an IOException.

Table 2.3

After you have created a socket, you can get the address and port details associated with the socket by the use of the following methods (Table 2.4).

Method	Description
InetAddress getAddress ()	Returns the address associated with the socket instance.
int getPort ()	Returns the remote port to which this socket is connected.
int getLocalPort ()	Returns the local port to which this socket instance is connected.

Table 2.4

Since TCP is a stream based protocol, every TCP socket is associated with input and output streams. It is noted that Remember there should be two ports for all TCP connections, a port on the remote machine and a port on the local machine, through which the client communicates. The local port number need not be specified because the TCP/IP software allocates these ports dynamically. These ports are called ephemeral ports because they exist only for the duration of a volatile client/server transaction. As discussed earlier, Java defines another class that must be used to create server applications. The ServerSocket class is used to create servers that listen for either local or remote client programs to connect to them on standard ports

Constructor	Description
ServerSocket (int port) throws IOException	This constructor is used to create a server socket on the specified port with a default queue length of 50.
ServerSocket (int port, int queue length) throws IOException	This constructor is used to create a server socket on the specified port with the specified queue length. It can throw an IOException.

Table 2.5

2.3.1 Writing Client / Server

Systems using TCP To write a Server program:

- Step 1 : Create the server socket and begin listening.
- Setp 2 : Call the accept() method to get new connections.
- Setp 3 : Create input and output streams for the returned socket.
- Step 4 : Conduct the conversation based on the agreed protocol.
- Step 5 : Close the Client Stream and the Socket.
- Step 6 : Go back to step 2 or continue to step 7.
- Step 7 : Close the Server Socket.

Program 1:

For using TCP for a server program which sends the current data to the client upon its request and terminates when the client wants to quit.

```
import java.io.*; import
java.net.*; import
java.util.*; class TCPServer
{   public static void main (String args[
])
{
    ServerSocket server;
    recSocket;
    PrintWriter on;
    BufferedReader i; String msg; server
    = new ServerSocket (8001); for (; ;)
    {
        recSocket = Server.accept( );    i = new
        BufferedReader (new InputStreamReader
        (recSocket.getInputStream( )));    on = new
```

```

        PrintWriter (new OutputStreamWriter
        (recSocket.getOutputStream(
        am( )));    on.println
        (server);    on.flush( );
        msg = i.readLine( );
        if (msg.equals ("Date"))
            msg = "Client asked for "+msg+" and the
        current date
        is "+ (new Date()).toString( );    else if
        (msg.equals
        ("Quit"))
            System.exit(1);
        else msg = "Invalid
        Request"; on.println
        (msg); on.flush(
        );
            i.close( ); on.close(
            ); recSocket.close(
            );
    }
} }

```

To write a Client Program:

Setp 1 : Create the Client Socket Connection.

Step 2 : Acquire read and write streams for the socket.

Step 3 : Use the streams according to the server.s protocol.

Step 4 : Close the streams.

Step 5 : Close the socket.

```

import java.io.* import
java.net.*;
class
TCPClie
nt
{    public static void main (String args[ ])throws
IOException
    {
        Socket
        socket;
        BufferedReader in;
        PrintWriter out; String
        request;
        socket = new Socket ("Vandana", 8001); out =
        new PrintWriter (new OutputStreamWriter

```

```

(Socket.getInputStream( ));
System.out.println ("Server says :"+
+in.readLine( ));
DataInputStream i = new
DataInputStream(System.in);
System.out.print ("Client can
Request for Date or quit :");
request = i.readLine( );
out.println
(request); out.flush(
);
System.out.println ("server responds:" +
in.readLine( )); in.close( ). out.close( );
socket.close( );    } }

```

2.3.2 Datagram packet

The java.net package provides two classes namely, DatagramPacket and DatagramSocket through which you can communicate with remote systems. A UDP Datagram is encapsulated in an instance of the class DatagramPacket. The DatagramPacket object is the container for the data while the DatagramSocket is used to send or receive the datagram packets. The class DatagramPacket defines two constructors which allow you to create datagram packet. The first constructor is used for receiving data over a datagram socket. DatagramPacket (byte buffer[], int size) The second constructor is used for transmitting datagrams. This constructor requires the destination machine's address and port number from the buffer and the size parameters.

DatagramPacket (byte buffer, int size, InetAddress destination, int port). Other methods defined in the class DatagramPacket are **(Table 2.6)**.

Method	Description
InetAddress getAddress()	Returns the destination machine's address.
int getPort()	Returns the portnumber.
byte[] getData()	Returns data contained in the datagram.
int getLength()	Returns the length of the valid data contained in the byte array returned from the getData() method.

Table 2.6

DatagramPacket class does not provide the mechanism of sending and receiving datagrams. The companion class DatagramSocket provides this functionality

Constructor	Description
DatagramSocket() throws SocketException	This constructor is used to create a socket at an unused port generally used for client applications.
DatagramSocket (int port) throws SocketException	This constructor is used to specify a particular port which is useful for server applications.
DatagramSocket (int port, InetAddress addr) throws SocketException	This constructor is used for machines having multiple IP addresses. You can use this constructor send and listen for datagrams from one of the IP addresses assigned to the machine.

Table 2.7.

2.3.3 Constructor Description

DatagramSocket() throws - This constructor is used to create a socket at an unused The important methods are shown in **Table 12.8.**

Method	Description
void send (DatagramPacket packet) throws IOException	This method sends the datagram's data to the previously defined host and port by taking a DatagramPacket object as a parameter.
void receive (DatagramPacket p) throw IOException	This method receives a datagram by taking a DatagramPacket object as a parameter. To extract data from the received datagram use the getData() method defined in the
DatagramPacket Class.	
void close ()	This method closes the opened datagram socket.

Table 12.8.

2.3.4 Server and client Program

This program displays the local time at the specified web site where the web site.s name is typed at the command line.

```
import java.net.*;
import java.io.*; class daytimedemo
{
    final static int DAY_TIME_PORT = 13;
    public static void main (String args[ ])
    {
        DatagramSocket socket;
        InetAddress Destination;
        DatagramPacket datagram;
        byte dataSend[ ] = new byte
        [256]; String dataReceive;
        if
        (args.length == 0)
        {
```

```

        System.out.println ("Usage: Java Daytime
Demo");    System.exit (0);
    } try
    {
        socket = new DatagramSocket(
    );
        Destination =
        InetAddress.getByName(args[0]);
        datagram = new DatagramPacket
        datasend, datasend.Length,
        destination,
        DAY_TIME_PORT);    socket.send (datagram);
        datagram = new DatagramPacket    datasend,
        datasend.length);    socket.receive
        (datagram);    datareceive = new
        String(datagram.    getData(
    ));
        System.out.println (The time at the
site"+destination.getHostNane( ) + "is"
        + dataReceive);
        Socket.close( );
    } catch (UnknownHostException
h
    )
    {
        System.out.println ("The website is
unknown");
    } catch (IOException
I)
    {
        System.out.println ("Error in sending or
receiving    datagram");
    } catch (socketException
s
    )
    {
        System.out.println("Socket cannot be
created"); }

```

2.4. Network

Given the five classes, InetAddress, Socket, ServerSocket, DatagramSocket and DatagramPacket, you can program any Internet protocol in existence. They also give you powerful low level control over Internet connections.

2.5 Learner Activity

1. What is networking?

2. Define the Networking Classes.
3. What is the Datagram Packet?
4. What is InetAddress Classes?

2.6 Coding Activity

The task is for students to implement a basic TCP-based chat system where the client sends a message to the server, and the server responds.

Instructions for Students:

1. Compile and Run the Server
2. Compile and Run the Client
3. The client sends "Hello, Server!" to the server.
4. The server receives the message and responds with "Message received: Hello, Server!".

Chapter Three: JAVA DATABASE CONNECTIVITY



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Features of the JDBC API
- The JDBC-ODBC Bridge
- JDBC Interface Levels
- The DriverManager Class
- Java.SQL Package
- The Application Layer

INTRODUCTION

JDBC is not a derivative of Microsoft's open Database Connectivity specification (ODBC). JDBC is written entirely in java and ODBC is a C interface. While ODBC is usable by non C languages like Visual Basic, it has the inherent development risks of C, like memory leaks. However, both JDBC and ODBC are based on the X/oPen SQL command Level Interface (CLI). Having the same conceptual base allows work on the API to proceed quickly and makes acceptance and learning of the API easier. Sun provides a jdbc - odbc bridge that translates JDBC - ODBC. This implementation when done with native methods, is very small and efficient.

3.1 Features Of The Jdbc API

The JBDC API was first introduced with release 1.1. of the JDK. JDK 1.4 contains JDBC 3.0, which is composed of the java.sql and javax.sql packages.

JDBC provides application developers with a single API that is uniform and database independent. The API provides a standard to write to, as well as a standard that takes all of the various application designs into account. The API's database independence is due to a set of Java interfaces that are implemented by a driver. The driver takes care of translating the standard JDBC calls into the specific calls required by the database it supports. The application is written once and then moved to the various drivers. The application remains the same; the drivers change. Drivers may be used to develop the middle tier of a multitier database design.

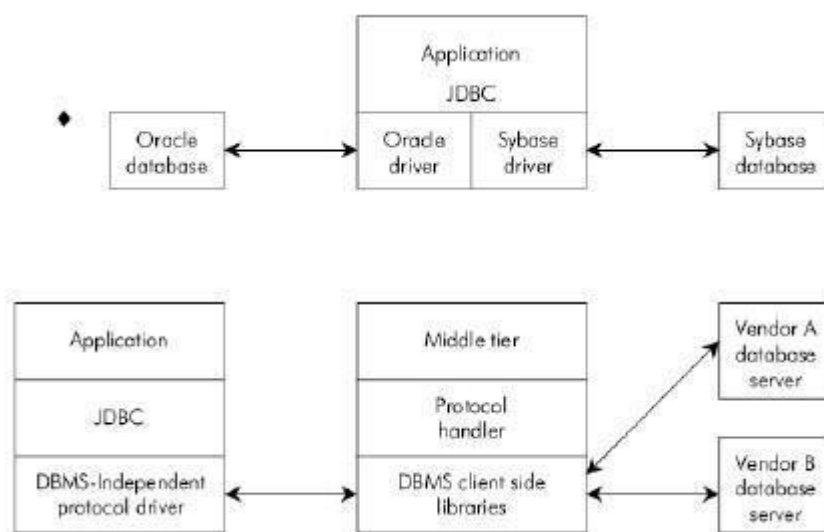


Figure 3.1: JDBC Database Designs

In addition to providing developers with a uniform and database independent framework, JDBC also provides a means of allowing developers to retain the specific functionality that their database vendor offers. JDBC drivers must support ANSI SQL- 2 Entry Level but JDBC allows developers to pass query strings directly to the connected driver. These strings may or may not be ANSI SQL, or SQL at all. The user of these strings is up to the underlying driver. (Of course, using this feature limits your freedom to change database back ends.) Every Java client or J2EE application that uses JDBC must have at least one JDBC driver, and each driver is specific to the type of DBMS used. A driver does not, however, need to be directly associated with a database. JDBC is not a derivative of Microsoft's Open Database Connectivity (ODBC) specification.

JDBC is written entirely in Java; ODBC is a C interface. Both JDBC and ODBC, however, are based on the

X/Open SQL Command Level Interface (CLI). JavaSoft provides a JDBCODBC bridge that translates JDBC to ODBC. This implementation, done with native methods, is very small and efficient. The JDBCODBC bridge is discussed later in the unit.

3.1.1 THE JDBC-ODBC BRIDGE

Connectivity

The JDBC-ODBC Bridge is a JDBC driver that provides translation of JDBC calls to ODBC operations. There are a number of DBMSs that support ODBC. When a company the size of Microsoft creates a standard for database access, there are sure to be vendors that follow; in fact, there are more than 50 different ODBC drivers available. As mentioned earlier in this unit, both JDBC and ODBC are based on the X/Open CLI, so the translation between JDBC and ODBC is relatively straightforward. ODBC is a client-side set of libraries and a driver that is specific to the client's operating system and, in some cases, machine architecture. From the developer's perspective, using a JDBC-ODBC bridge driver is an easy choice. Applications will still speak directly to the JDBC interface classes, so it is exactly the same as using any other JDBC driver; however, the implementation of a JDBC-ODBC bridge requires that the developer be aware of what is required to run the application. Because ODBC calls are made using binary C calls, the client must have a local copy of the ODBC driver, the ODBC driver manager, and the client-side libraries. The JDBC-ODBC bridge is not the best choice for portability or performance, although it may be an acceptable and simple driver when developing test systems and prototypes.

3.2 JDBC Interface Levels

There are two JDBC interface levels:

- The application layer is where the developer uses the API to make calls to the database via SQL and retrieve the results.
- The Driver layer handles all communication with a specific driver implementation.

Figure 13.2 illustrates the Driver and Application layers.

Fortunately, the application developer only needs to use the standard API interfaces to guarantee JDBC compliance. The driver developer is responsible for developing code that interfaces to the database and supports the JDBC application level calls. It is important, however, to understand the Driver layer, and how some of the objects that are used at the Application layer are created by the driver. There are four main interfaces that every driver

layer must implement, and one class that bridges the Application and Driver layers. The four interfaces are Driver, connection, Statement and ResultSet. The Driver interface implementation is where the connection to the database is made. In most applications, the Driver is accessed through the DriverManager class-providing one more layer of abstraction for the developer. The Connection, Statement, and ResultSet interfaces are implemented by the driver vendor, but these interfaces specify the methods that the application developer can use.

They allow the developer to create statements and retrieve results without having to think about where the objects are coming from or worry about what specific driver the application will use. The following sections discuss the Driver and Application layers in more detail.

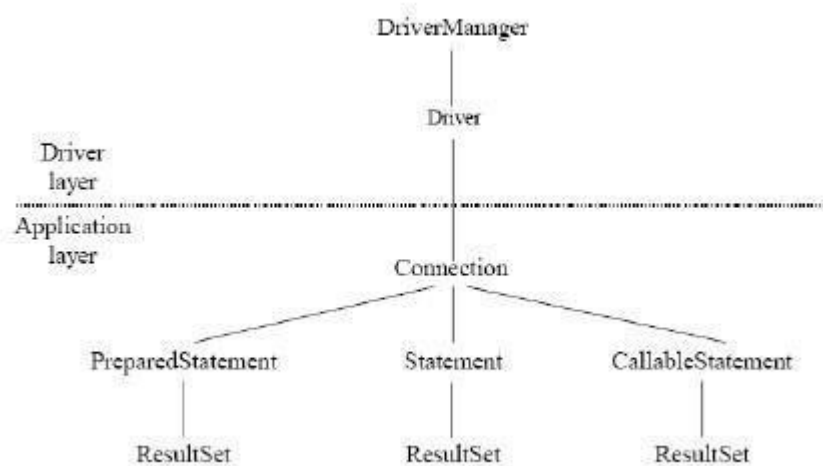


Figure 13.2: JDBC API Components

3.3 The DriverManager Class:

The DriverManager Class is actually a utility class used to manage JDBC drivers. The class provides methods to obtain a connection through a driver, register and deregister drivers, set up logging, and set login timeouts for database access. The important DriverManager methods are listed below. Because they are static, they may be referenced through the interface. SQL Exception is thrown if there is trouble with the remote database.

- a) Public Static Synchronized Connection getConnection (String url, properties info) throws SQLException.**

Attempts to return a reference to an object implemented from the Connection interface. The method sweeps through a vector of stored Driver classes, passing the URL string and

Properties object info to each in turn. The first Driver class that returns a Connection is used, info are a reference to a Properties container object of tags/value pairs typically username/password. The method allows several attempts to make an authorized connection for each driver in the vector.

b) Public static synchronized Connection getConnection (String url) throws SQLException

Calls getConnection (url, info) with an empty Properties object (info). Public static synchronized Connection getConnection (String url, String user, String password) Connectivity throws SQLException. Creates a Properties object (info), stores the user and password strings into it, and then calls getConnection (url, info).

c) Public static synchronized void registerDriver (java.sql.Driver driver) throws SQLException

Stores

the instance of the Driver interface implementation into a vector of drivers, along with an instance of securityContext, which identifies where the driver came from.

d) Public static void setLogWriter (java.io.PrintWriter out)

Sets a private static java.io.PrintWriter reference to the PrintWriter object passed to the method.

3.4 Java.Sql Package

Many J2EE servers support distributed transactions that involve multiple databases. For example, you could run a SQL statement on a Connection from one DataSource for an Oracle database, and run a SQL statement on a Connection from a DataSource for a SQL Server database. If your J2EE server supports distributed transactions, you can specify that those two SQL statements are in the same transaction, and must commit or rollback together. JDBC 2.0 added an interface called XADataSource to allow JDBC drivers to support distributed transactions. As with the ConnectionPoolDataSource, you should not need to directly use the XADataSource class. If your JDBC driver and J2EE server support distributed transactions, they should use this interface behind the scenes. You don't have to write any special code for this purpose.

3.4.1 The Application Layer

The Application layer encompasses three interfaces that are implemented at the Driver layer, but are used by the application developer. In Java, the interface provides a means of using a general type to indicate a specific class. The interface defines methods that must be implemented by the specific classes. For the application developer, this means that the specific Driver class implementation is irrelevant; simply coding to the standard JDBC APIs will be sufficient. (Of course, this is assuming that the driver is JDBCcompliant, which means that the database is at least ANSI SQL-2 Entry Level). The three main interfaces are Connection, Statement, and ResultSet. A Connection object is obtained from the driver implementation through the DriverManager.getConnection () method call. After a Connection object is returned, the application developer may create a Statement object to issue SQL against the database. If the SQL that was submitted was a SELECT query, the result set is returned in a ResultSet object.

3.4.2 The Connection Interface

The Connection interface represents a session with the database connection provided by the Driver. Typical database connections include the ability to control changes made to the actual data stored through transactions. On creation, JDBC connections are in an auto-commit mode. There is no rollback possible. After getting a Connection object from the driver, the developer should consider setting auto commit to false with the SetAutoCommit (Boolean b) method. When auto-commit is disabled, the Connection will support both Connection.commit() and Connection.rollback() method calls. The level of support for transaction isolation depends on the underlying support for transactions in the database. A portion of the connection interface definition follows. As usual, SQLException is thrown in case of trouble with the remote database.

a) Statement createStatement () throws SQLException.

The Connection object implementation will return an instance of an implementation of a Statement object.

The Statement object is then used to issue queries.

b) PreparedStatement prepareStatement (String sql) throws SQL Exception

The Connection object implementation will return an instance of a PreparedStatement object that is configured with the SQL string passed. The driver may then send the

statement to the database, if the database (driver) handles precompiled statements; otherwise, the driver may wait until the PreparedStatement is executed by an execute method.

c) CallableStatement prepareCall (String sql) throws SQLException

The Connection object implementation will return an instance of a CallableStatement. Callable Statements are optimized for handling stored procedures. The driver implementation may send the sql string immediately when prepareCall () is complete or may wait until an execute method occurs.

d) void setAutoCommit (Boolean autoCommit) throws SQLException

Sets a flag in the driver implementation that enables commit/rollback (false) or makes all transactions commit immediately (true).

e) void commit () throws SQLException

Makes all changes made since the beginning of the current transaction (either since the opening of the Connection or since the last commit () or roll back ()).

f) void rollback () throws SQLException

Drops all changes made since the beginning of the current transaction. The primary use of the Connection interface is to create a statement:

```
Connection conn; Statement  
stmt; conn = DriverManager.getConnection  
(url); Connectivity  
stmt = c.createStatement ( );
```

This statement may be used to send SQL statements that return a single result set in a ResultSet object reference. Statements that need to be called a number of times with slight variations may be executed more efficiently using a PreparedStatement. The Connection interface is also used to create a CallableStatement whose purpose is to execute stored procedures. Most of the time, the developer knows the database schema before hand and creates the application based on the schema; however, JDBC provides an interface that may be used to dynamically determine the schema of a database. The Connection interface getMetaData () method will return a DatabaseMetaData object. The instance of the class that implements the interface provides information about the database as a whole, including access information about tables and procedures, column names, data types, and

so on. The implementation details of DatabaseMetaData are dependent on the database vendor's ability to return this type of information.

3.4.3 The Statement Interface

A statement is the vehicle for sending SQL queries to the database and retrieving a set of results. Statements can be SQL updates, inserts, deletes, or queries; statements may also create or drop tables. The Statement interface provides a number of methods designed to ease the job of writing queries to the database. The important Statement methods are listed below. As usual, SQLException is thrown if there is a problem with the remote database.

a) ResultSet executeQuery (String sql) throws SQLException

Executes a single SQL query and returns the results in an object of type ResultSet.

b) int executeUpdate (String sql) throws SQLException

Executes a single SQL query that returns a count of rows affected rather than a set of results.

c) boolean execute (String sql) throws SQLException

A general way to execute SQL statements that may return multiple result sets and/or update counts. This method is also used to execute stored procedures that return out and inout parameters. The getResultSet (), getUpdateCount (), and getMoreResults () methods are used to retrieve the data returned. (The method is less likely to be used than executeUpdate () and executeQuery ().)

d) ResultSet getResultSet () throws SQLException

Returns the result of a statement execution as a ResultSet object. Note that if there are no results to be read, or if the result is an update count, this method returns null. Also note that once read, the results are cleared.

e) int getUpdateCount () throws SQLException

Returns the status of an Update, an Insert, or a Delete query; a stored procedure; or a DDL statement. The value returned is the number of rows affected. -1 is returned if there is no update count, or if the data returned is a result set. Once read, the update count is cleared.

f) boolean getMoreResults () throws SQLException

Moves to the next result in a set of multiple results/update counts. This method returns true if the next result is a ResultSet object. This method will also close any previous

g) ResultSet read.

Statement methods may or may not return a ResultSet object, depending on the Statement method used.

The executeUpdate () method, for example, is used to execute SQL statements that do not expect a result

(except a row-count status); int rowCount;rowCount = stmt.executeUpdate (".DELETE FROM Customer WHERE Cusotier ID = .McG10233. ."); SQL statements that return a single set of results can use the executeQuery () method. This method returns a single Result Set object. The object represents the row information returned as a result of the query:

```
ResultSet results;  
results = stmt.executeQuery (".SELECT * FROM Stock.");
```

SQL statements that execute stored procedures (or trigger a stored procedure) may return more than one set of results. The execute () method is a general-purpose method that can return either a single result set or multiple result sets. The method returns a Boolean flag that is used to determine whether there are more result sets.

Because a result set could contain either data or the count of an operation that returns a row count, the getResultSet (), getMoreResults (), and getUpdateCount () methods are used. Here is an example:

```
//Assume SQLString returns multiple  
result sets //returns true if a  
ResultSet is returned int count;  
ResultSet results; If (stmt.execute  
(SQL string))  
{  
results = stmt.getResultSet ( );  
// false, an UpdateCount was  
returned  
} else  
{ count = stmt.getUpdateCount ( );  
};  
}  
// Process the first results here....  
Connectivity
```

```

// Now loop until there are no more
//results or update counts
do {
// Is the next result a ResultSet?
If (stmt.getMoreResults())
{ results = stmt.getResultSet
  };
} else {
count =
stmt.getUpdateCount();
}
// Process next results (if any) here....
} while ((results !=null) &&
(count != 1);

```

3.4.4 The PreparedStatement Interface

The PreparedStatement interface extends the Statement interface. When there is a SQL statement that requires repetition with minor variations, the PreparedStatement provides the mechanism for passing a precompiled SQL statement that uses parameters. Public interface PreparedStatement extends Statement PreparedStatement parameters are used to pass data into a SQL statement, so they are considered in parameters and are filled in by using setType methods:

```

// Assume pricelist is an array of prices
that needs
// to be reduced for a 10% off sale, and
reduced items
// is an array of item IDs int
reduction = 10;
PreparedStatement ps = conn.prepareStatement
(*UPDATE Catalog SET Price = WHERE ItemID =7")
// Do the updates in a loop
for (int I = 0; I < reducedItems.length; i++)
{
//Note that the setType methods set the value of
the
//parameters noted in the SQL statement
with question //marks (?). They are
indexed, starting from 1 to n.
ps.setFloat (1.
(pricelist[i]*((float)(100=reduction)/100)));
ps.setString(2, reducedItems[i]); if
(ps.executeUpdate() == 0)
{

```

```
throw new ApplicationSpecificException  
(*No Item ID: " + reduced Items[i]);
```

Parameters hold their current values until either a new setType method is called, or the method clearParameters() is called for PreparedStatement object. In addition to the execute methods inherited from Statement, PreparedStatement declares the setType methods listed in **Table 3.1**. Each method takes two arguments a parameter index, and the primitive or class type, as shown in **table 3.1**.

Method signature	JAVA Type	SQL Type form Database
void setBigDecimal (int index, BigDecimal x)	java.math.BigDecimal	NUMERIC
void setBoolean (int index, boolean b)	Boolean	BIT
void setByte (int index, byte b)	Byte	TINTINT
void setBytes (int index, byte x)	byte array	BINARY or VARBINARY
void setDate (int index, Date d)	java.sql.Date	DATE
void setDouble (int index, double d)	Double	DOUBLE
void setFloat (int index, float f)	Float	FLOAT
void setInt (int index int i)	Int	INTEGER
void setLong (int index long l)	Long	BIGINT
void setNull (int index, int sql type)		java, sql Type lists SQL Types by number, and null is integer 0 (zero)
void setShort (int index, short x)	Short	SMALLINT
void setString (int index, String s)	java.lang.String	VARCHAR or LONGVAR CHAR
void setTime (int index, Time t)	java.sql.Time	TIME
void setTimestamp (int index, Timestamp st)	java.sql.Timestamp	TIMESTAMP
void setAsciiStream (int index, InputStream istr, int length)	InputStream	LONGVARCHAR
void setBinaryStream (int index,InputStream istre, int length)	InputStream	LONGVARBINERY
void setUnicodeSteam (int index, InputStream istre, int length)	InputStream	LONGVARCHAR

Table 3.1: The Set Type Methods

3.4.5 The Callable Statement Interface

The CallableStatement interface is used to execute SQL stored procedures. CallableStatement inherits from the PreparedStatement interface, so all of the execute and setType methods are available. The syntax of stored procedures varies among database vendors, so JDBC defines a standard way to call stored procedures in all RDBMSs.

a) public interface CallableStatement extends PreparedStatement

The JDBC uses an escape syntax that allows parameters to be passed as in parameters and out parameters. The syntax also allows a result to be returned. If this syntax is used, the parameter must be registered as an out parameter. Method Signature Java Type SQL Type from the Database Here is an example of a CallableStatement with an out parameter.

```
Connectivity
CallableStatement cs =
conn.prepareCall ("[ca]] GetQuote
```

```

(?, ?)}"); cs.setString (1,
stockName);
// java.sql.Types defines SQL data
types that are returned // as out
parameters cs.registerOutParameter(2,
Types, FLOAT); cs.executeUpdate
(); float quote = cs.getFloat
(2);

```

CallableStatement defines a set of get Type methods that convert the SQL types returned from the database to Java types. These methods match the setType methods declared by Prepared Statement, and are listed in Table 3.2

Method signature	JAVA Type	SQL Type form Database
BigDecimal getBigDecimal (int index, int scale)	java.math.BigDecimal	NUMERIC
boolean getBoolean (int index)	Boolean	BIT
byte getByte (int index)	Byte	TINTINT
byte[] getBytes (int index)	byte array	BINARY or VARBINARY
Date getDate (int index)	java.sql.Date	DATE
Double getDouble (int index)	Double	DOUBLE
float getFloat (int index)	Float	FLOAT
int getInt (int index)	Int	INTEGER
long getLong (int index)	Long	BIGINT
short getShort (int index)	Short	SMALLINT
string getString (int index)	String	CHAR, VARCHAR or LONGVAR CHAR
Time getTime (int index)	java.sql.Time	TIME
Timestamp getTimestamp (int index)	java.sql.Timestamp	TIMESTAMP

Table 3.2: The Get Type Methods

3.4.6 The ResultSetInterface

The ResuletSet interface defines methods for accessing tables of data generated as the result of executing a Statement. ResultSet column values may be accessed in any order-they are indexed and may be selected by either the name or the number (from 1 to n) of the column. ResultSet maintains the position of the current row, starting with the first row of data returned. The next () method moves to the next row of data. A partial look at the ResultSet interface follows. Vendors that implement this interface have the option of catching the results on the remote side, so there is the possibility of a communication problem.

For this reason, the following methods listed below SQLException.

a) boolean next() throws SQLException .

Positions the ResultSet to the next row. The ResultSet row position is initially just before the first row of the result set. Method Signature Java Type SQL Type from The Database

b) ResultSet MetaData getMetaData() throws SQLException

Returns an object that contains a description of the current result set, including the number of columns, the type of each column, and properties of the results.

c) void close() throws SQLException

Normally a ResultSet is closed when another SQL statement is executed, but it may be desirable to release the resources earlier. As with the CallableStatement interface discussed earlier, the resulting data can be read through getType () methods. Note, however, that values can be accessed either via column names or column numbers; callable statements can use only column numbers. Recall that column numbers begin at

1.

3.4.7 The ResultSet MetaData Interface

Aside from being able to read data from a ResultSet object, JDBC provides an interface to allow the developer to determine what type of data was returned. The ResultSetMetaData interface is similar to the DatabaseMetaData interface in concept, but it is specific to the current ResultSet. As with Database, MetaData, it is unlikely that many developers will use this interface because most applications are written with an understanding of the database schema and column names and values; however, ResultSetMetaData is useful in dynamically determining the MetaData of a ResultSet returned from a stored procedure.

3.4.8 Scrollable or Updatable ResultSets

By default, when you execute a Statement that creates a ResultSet, the ResultSet is read-only and the rows can only be read in order; however, JDBC 2.0 and above drivers allow you the option of making a ResultSet that is updatable or scrollable. To do this, you must supply two additional parameters to the Connection interface's createStatement, prepareStatement, or prepareCall methods.

- Statement `createStatement (String sql, int result-SetType, int resultSetConcurrency)` throws `SQLException`
- `PreparedStatement prepareStatement (String sql, int ResultSetType, int resultSetConcurrency)` **throws SQL Exception.**
- The `resultSetType` parameter can be one of the following three values:
 - `ResultSet TYPE FROWARD ONLY`
- This is the default value; it creates a forward only `ResultSet`.

a) `ResultSet.TYPE_SCROLL_INSENSITIVE`

Creates a forward and backward scrollable `ResultSet` that will not reflect any changes to the database data after the query data is returned.

b) `ResultSet.TYPE_SCROLL_SENSITIVIES`

Created a forward and backward scrollable `ResultSet` that will be updated if the database Connectivity data changes even after the query data is returned. The `resultSetConcurrency` parameter can be one of the following two values:

c) `ResultSet.CONCUR_READ_ONLY`

Creates an updatable `ResultSet` that allows updates insertions, and deletions, and backward scrollable `ResultSet`:

```
Statement stmt;
stmt = conn.createStatement
(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
ResultSet results;
Results = stmt.executeQuery ("SELECT * FROM
Stock");
```

If the `ResultSet` is scrollable, in addition to `next()`, you may use `previous()`, `first()`, `last()`, `absolute(int)`, and `relative(int)` methods to move around the `ResultSet`.

LEARNER ACTIVITY 3

1. Define JDBC.
2. Describe JDBC/ODBC Bridge.
3. Describe the driver manager class.

4. Describe the connection Interface, Statement Interface and Resultset Interface.
5. Describe ResultSet Metadata Interface.

CODING ACTIVITY

Students will learn how to connect a Java program to a MySQL database using JDBC. They will perform basic CRUD (Create, Read, Update, Delete) operations.

Chapter Four: SERVLETS



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Features of the JDBC API
- HTTP Servlets
- Servlet Life Cycle
- The Servlet API Packages

INTRODUCTION

One of the most essential technologies in J2EE is Servlets. Servlets provide the best mechanism for J2EE applications to provide a web interface. This chapter about the essential concepts of developing servlets. In 1993, messages sent in Hypertext Transfer Protocol (HTTP) format used by web servers were only a minute portion of the traffic on the Internet. HTTP's tremendous functionality led a rapid expansion in both the number and capabilities of web servers. Essentially, it is web HTTP that made the world wide web possible. Originally, a basic web server simply returned a text page when it got correctly formatted request message; however it did not take the programmer long to discover that good things were possible if some extra functionality was added, such as forms and graphics and capabilities. The Common Gateway Interface (CGI) standard enables the user of a web browser to submit data from a web page to a server for further processing. Programming with Java servlets and Java server pages builds on this universal standard and

makes it easier to me. To understand how this works, you must understand how a web server processes. HTTP servlet Application programming Interface (API).

4.1 Servlets

A servlet is a Java component that can be plugged into a Java-enabled web server to provide custom services. These services can include:

- New features
- Runtime changes to content
- Runtime changes to presentation
- New standard protocols (such as FTP)
- New custom protocols

Servlets are designed to work within a request/response processing model. In a request/response model, a client sends a request message to a server and the server responds by sending back a reply message. Requests can come in the form of an

- HTTP
- URL,
- FTP,
- URL, or a custom protocol.

The request and the corresponding response reflect the state of the client and the server at the time of the request. Normally, the state of the client/server connection cannot be maintained across different request/response pairs. However, session information is maintainable with servlets through means to be described later. The Java Servlet API includes several Java interfaces and fully defines the link between a hosting server and servlets. The Servlet API is defined as an extension to the standard JDK. JDK extensions are packaged under javax--the root of the Java extension library tree. The Java Servlet API contains the following packages: Package javax.servlet • Package javax.servlet.http

Servlets are a powerful addition to the Java environment. They are fast, safe, reliable, and 100% pure Java. Because servlets plug into an existing server, they leverage a lot of existing code and technology. The server handles the network connections, protocol negotiation,

class loading, and more; all of this work does not need to be replicated! And, because servlets are located at the middle tier, they are positioned to add a lot of value and flexibility to a system. In this course you will learn about the Servlet API and you will get a brief tour of the types of features servlets can implement.

4.1.1 Architectural Roles for Servlets

Because of their power and flexibility, servlets can play a significant role in a system architecture. They can perform the application processing assigned to the middle tier, act as a proxy for a client, and even augment the features of the middle tier by adding support for new protocols or other features. A middle tier acts as the application server in so called three-tier client/server systems, positioning itself between a lightweight client like a web browser and a data source.

4.1.2 Middle-Tier Process

In many systems a middle tier serves as a link between clients and back-end services. By using a middle tier a lot of processing can be off-loaded from both clients (making them lighter and faster) and servers (allowing them to focus on their mission).

One advantage of middle tier processing is simply connection management. A set of servlets could handle connections with hundreds of clients, if not thousands, while recycling a pool of expensive connections to database servers.

Other middle tier roles include: •

- Business rule enforcement

- Transaction management
- Mapping clients to a redundant set of servers
- Supporting different types of clients such as pure HTML and Java capable clients

4.1.3 Proxy Servers

When used to support applets, servlets can act as their proxies. This can be important because Java security allows applets only to make connections back to the server from which they were loaded. If an applet needs to connect to a database server located on a different machine, a servlet can make this connection on behalf of the applet.

4.1.4 Protocol Support

The Servlet API provides a tight link between a server and servlets. This allows servlets to add new protocol support to a server. (You will see how HTTP support is provided for you in

the API packages.) Essentially, any protocol that follows a request/response computing model can be implemented by a servlet. This could include:

- SMTP
- POP
- FTP

Servlet support is currently available in several web servers, and will probably start appearing in other types of application servers in the near future. You will use a web server to host the servlets in this class and only deal with the HTTP protocol. Because HTTP is one of the most common protocols, and because HTML can provide such a rich presentation of information, servlets probably contribute the most to building HTTP based systems.

4.1.4 HTML Support

HTML can provide a rich presentation of information because of its flexibility and the range of content that it can support. Servlets can play a role in creating HTML content. In fact, servlet support for HTML is so common, the `javax.servlet.http` package is dedicated to supporting HTTP protocol and HTML generation. Complex web sites often need to provide HTML pages that are tailored for each visitor, or even for each hit. Servlets can be written to process HTML pages and customize them as they are sent to a client. This can be as simple as on the fly substitutions or it can be as complex as compiling a grammar-based description of a page and generating custom HTML.

4.1.5 Inline HTML Generation

Some web servers, such as the Java Web Server (JWS), allow servlet tags to be embedded directly into HTML files. When the server encounters such a tag, it calls the servlet while it is sending the HTML file to the client. This allows a servlet to insert its contribution directly into the outgoing HTML stream.

4.1.6 Server-Side Includes

Another example is on the fly tag processing known as server-side includes (SSI). With SSI, an HTML page can contain special commands that are processed each time a page is requested. Usually a web server requires HTML files that incorporate SSI to use a unique extension, such as `.shtml`. As an example, if an HTML page (with an `.shtml` extension) includes the following:

```
<!--#include virtual="/includes/page.html"-->
```

it would be detected by the web server as a request to perform an inline file include. While server side includes are supported by most web servers, the SSI tags are not standardized. Servlets are a great way to add server side include processing to a web server. With more and more web servers supporting servlets, it would be possible to write a standard SSI processing servlet and use it on different web servers.

4.1.7 Replacing CGI Scripts

An HTTP servlet is a direct replacement for Common Gateway Interface (CGI) scripts. HTTP servlets are accessed by the user entering a URL in a browser or as the target of an HTML form action. For example, if a user enters the following URL into a browser address field, the browser requests a servlet to send an HTML page with the current time: `http://localhost/servlet/DateTimeServlet` The `DateTimeServlet` responds to this request by sending an HTML page to the browser.

Note that these servlets are not restricted to generating web pages; they can perform any other function, such as storing and fetching database information, or opening a socket to another machine.

4.1.8 Installing Servlets

Servlets are not run in the same sense as applets and applications. Servlets provide functionality that extends a server. In order to test a servlet, two steps are required: Install the servlet in a hosting server Request a servlet's service via a client request

There are many web servers that support servlets. It is beyond the scope of this course to cover the different ways to install servlets in each server. This course examines the JSDK's `servletrunner` utility and the JWS.

4.1.9 Temporary versus Permanent Servlets

Servlets can be started and stopped for each client request, or they can be started as the web server is started and kept alive until the server is shut down. Temporary servlets are loaded on demand and offer a good way to conserve resources in the server for less-used functions. Permanent servlets are loaded when the server is started, and live until the server is shutdown. Servlets are installed as permanent extensions to a server when their

start-up costs are very high (such as establishing a connection with a DBMS), when they offer permanent server-side functionality (such as an RMI service), or when they must respond as fast as possible to client requests. There is no special code necessary to make a servlet temporary or permanent; this is a function of the server configuration. Because servlets can be loaded when a web server starts, they can use this auto-loading mechanism to provide easier loading of serverside Java programs. These programs can then provide functionality that is totally unique and independent of the web server. For example, a servlet could provide R-based services (rlogin, rsh, ...) through TCP/IP ports while using the servlet request/response protocol to present and process HTML pages used to manage the servlet.

4.1.10 Using `servletrunner`

For both JDK 1.1 and the Java 2 platform, you need to install the Java Servlet Development Kit (JSDK). To use `servletrunner`, make sure your `PATH` environment variable points to its directory. For the JSDK 2.0 installed with all default options, that location is: `c:\jsdk2.0\bin` on a Windows platform.

To make sure that `servletrunner` has access to the Java servlet packages, check that your `CLASSPATH` environment variable is pointing to the correct JAR file, `c:\jsdk2.0\lib\jsdk.jar` on a Windows platform. With the Java 2 platform, instead of modifying the `CLASSPATH`, it is easier to just copy the JAR file to the `ext` directory under the Java runtime environment. This treats the servlet packages as standard extensions.

With the environment set up, run the `servletrunner` program from the command line. The parameters are:

Usage: `servletrunner [options]` Options:

- p port** the port number to listen on
- b backlog** the listen backlog
- m max** maximum number of connection handlers
- t timeout** connection timeout in milliseconds
- d dir** servlet directory
- s filename** servlet property file name

The most common way to run this utility is to move to the directory that contains your servlets and run `servletrunner` from that location. However, that doesn't automatically configure the tool to load the servlets from the current directory.

4.1.11 Using Java Web Server

Sun's Java Web Server (JWS) is a full featured product. For servlet developers, a nice feature is its ability to detect when a servlet has been updated. It detects when new class files have been copied to the appropriate servlet directory and, if necessary, automatically reloads any running servlets.

The JWS can be installed as a service under Windows NT. While this makes it convenient for running a production server, it is not recommended for servlet development work. Under Windows 95, there are no OS services, so the command line start-up is your only option. To run JWS from the c:\JavaWebServer1.1\bin directory, type in the httpd command. This starts the server in a console window.

No further display is shown in the console unless a servlet executes a `System.out.println()` statement.

Servlets are installed by moving them to the c:\JavaWebServer1.1\servlets directory. As mentioned, JWS detects when servlets have been added to this directory. Although you can use the JWS management applet to tailor the servlet installation, this is generally not advised except for production server installations. To shut down the JWS, press <Control>+C in the command window. The server prints a message to the console when it has finished shutting down.

4.1.12 Servlet API

The Java Servlet API defines the interface between servlets and servers. This API is packaged as a standard extension to the JDK under javax:

- Package javax.servlet
- Package javax.servlet.http

The API provides support in four categories:

- Servlet life cycle management
- Access to servlet context
- Utility classes
- HTTP-specific support classes

4.1.13 The Servlet Life Cycle

Servlets run on the web server platform as part of the same process as the web server itself. The web server is responsible for initializing, invoking, and destroying each servlet instance. A web server communicates with a servlet through a simple interface, `javax.servlet.Servlet`. This interface consists of three main methods:

`init()`

`service()`

`destroy()`

and two

ancillary

methods:

`getServletConfig()`

`getServletInfo()`

You may notice a similarity between this interface and that of Java applets. This is by design! Servlets are to web servers what applets are to web browsers. An applet runs in a web browser, performing actions it requests through a specific interface. A servlet does the same, running in the web server.

a) The `init()` Method

When a servlet is first loaded, its `init()` method is invoked. This allows the servlet to perform any setup processing such as opening files or establishing connections to their servers. If a servlet has been permanently installed in a server, it loads when the server starts to run. Otherwise, the server activates a servlet when it receives the first client request for the services provided by the servlet. The `init()` method is guaranteed to finish before any other calls are made to the servlet--such as a call to the `service()` method. Note that `init()` will only be called once; it will not be called again unless the servlet has been unloaded and then reloaded by the server. The `init()` method takes one argument, a reference to a `ServletConfig` object which provides initialization arguments for the servlet. This object has a method `getServletContext()` that returns a `ServletContext` object containing information about the servlet's environment (see the discussion on Servlet Initialization Context below).

b) The service() Method

The service() method is the heart of the servlet. Each request message from a client results in a single call to the servlet's service() method. The service() method reads the request and produces the response message from its two parameters:

A ServletRequest object with data from the client. The data consists of name/value pairs of parameters and an InputStream. Several methods are provided that return the client's parameter information. The

InputStream from the client can be obtained via the getInputStream() method. This method returns a ServletInputStream, which can be used to get additional data from the client. If you are interested in processing character-level data instead of byte-level data, you can get a BufferedReader instead with getReader().

A ServletResponse represents the servlet's reply back to the client. When preparing a response, the method setContentType() is called first to set the MIME type of the reply. Next, the method getOutputStream() or getWriter() can be used to obtain a ServletOutputStream or PrintWriter, respectively, to send data back to the client.

As you can see, there are two ways for a client to send information to a servlet. The first is to send parameter values and the second is to send information via the InputStream (or Reader). Parameter values can be embedded into a URL. How this is done is discussed below. How the parameter values are read by the servlet is discussed later.

The service() method's job is conceptually simple--it creates a response for each client request sent to it from the host server.

However, it is important to realize that there can be multiple service requests being processed at once. If your service method requires any outside resources, such as files, databases, or some external data, you must ensure that resource access is thread-safe. Making your servlets thread-safe is discussed in a later section of this course.

c) The destroy() Method

The destroy() method is called to allow your servlet to clean up any resources (such as open files or database connections) before the servlet is unloaded. If you do not require any clean-up operations, this can be an empty method.

The server waits to call the destroy() method until either all service calls are complete, or a certain amount of time has passed. This means that the destroy() method can be called while some longerrunning service() methods are still running. It is important that you write

your `destroy()` method to avoid closing any necessary resources until all `service()` calls have completed.

Sample Servlet

The code below implements a simple servlet that returns a static HTML page to a browser.

This example fully implements the Servlet interface.

```
import java.io.*; import
javax.servlet.*; public class
SampleServlet implements Servlet
{      private ServletConfig
onfig; public void init
(ServletConfig config) throws
ServletException
    { this.config = config;
    }
    public void destroy() {} // do nothing
    public ServletConfig getServletConfig()
    {
        return
config;
    }      public
String
getServletInfo()
{      return "A
Simple Servlet";
    }
    public void service (ServletRequest
req,ServletResponse res)      throws
ServletException, IOException
    {      res.setContentType("text/html" );
PrintWriter out =
res.getWriter();
out.println( "<html>" );
out.println( "<head>" );
out.println( "<title>A Sample
Servlet</title>" );      out.println(
"</head>" );      out.println( "<body>"
);
out.println( "<h1>A Sample Servlet</h1>" );
out.println( "</body>" );
    out.println( "</html>" );
out.close();      }
}
```

4.1.14 Servlet Context

A servlet lives and dies within the bounds of the server process. To understand its operating environment, a servlet can get information about its environment at different times. Servlet initialization information is available during servlet start-up; information about the hosting server is available at any time; and each service request can contain specific contextual information.

4.1.15 Servlet Initialization Information

Initialization information is passed to the servlet via the `ServletConfig` parameter of the `init()` method. Each web server provides its own way to pass initialization information to a servlet. With the JWS, if a servlet class `DatePrintServlet` takes an initialization argument `timezone`, you would define the following properties in a `servlets.properties` file:

```
servlet.dateprinter.code=DatePrinterServlet  
servlet.dateprinter.timezone=PST
```

or this information could be supplied through a GUI administration tool. The `timezone` information would be accessed by the servlet with the following code:

```
String timezone;  
public void init(ServletConfig config) {  
    timeZone = config.getInitParameter("timezone");  
}
```

An Enumeration of all initialization parameters is available to the servlet via the `getInitParameterNames()` method.

4.1.16 Server Context Information

Server context information is available at any time through the `ServletContext` object. A servlet can obtain this object by calling the `getServletContext()` method on the `ServletConfig` object. Remember that this was passed to the servlet during the initialization phase. A well written `init()` method saves the reference in a private variable.

The *ServletContext* interface defines several methods. These are outlined below.

<code>getAttribute()</code>	An extensible way to get information about a server via attribute name/value pairs. This is server specific.
-----------------------------	--

getMimeType()	Returns the MIME type of a given file.
This method translates a relative or virtual path to a new path relative to the server's getRealPath() HTML documentation root location.	
getServerInfo()	Returns the name and version of the network service under which the servlet is running.
getServlet()	Returns a Servlet object of a given name. Useful when you want to access the services of other servlets.
getServletNames() Returns an enumeration of servlet names available in the current namespace.	
log()	Writes information to a servlet log file. The log file name and format are server specific.

The following example code shows how a servlet uses the host server to write a message to a servlet log when it initializes:

```
private ServletConfig config;
public void init(ServletConfig
config) {    // Store config in an
instance variable    this.config =
config;
    ServletContext sc = config.getServletContext();
    sc.log( "Started OK!" );
}
```

4.1.17 Servlet Context During a Service Request

Each service request can contain information in the form of name/value parameter pairs, as a ServletInputStream, or a BufferedReader. This information is available from the ServletRequest object that is passed to the service() method.

The following code shows how to get service-time information:

```
BufferedReader reader;
String param1;    String
param2;    public void
service          (
ServletRequest req,
    ServletResponse res) {
```

```

        reader = req.getReader();           param1
    = req.getParameter("First");           param2
    = req.getParameter("Second");       }

```

There are additional pieces of information available to the servlet through `ServletRequest`. These are shown in the following table.

<code>getAttribute()</code>	Returns value of a named attribute for this request.
<code>getContentTypeLength()</code>	Size of request, if known.
<code>getContentType()</code>	Returns MIME type of the request message body.
<code>getInputStream()</code>	Returns an <code>InputStream</code> for reading binary data from the body of the request message.
<code>getParameterNames()</code>	Returns an array of strings with the names of all parameters.
<code>getParameterValues()</code>	Returns an array of strings for a specific parameter name.
<code>getProtocol()</code>	Returns the protocol and version for the request as a string of the form <code><protocol>/<major version>.<minor version></code> .
<code>getReader()</code>	Returns a <code>BufferedReader</code> to get the text from the body of the request message.
<code>getRealPath()</code>	Returns actual path for a specified virtual path.
<code>getRemoteAddr()</code>	IP address of the client machine sending this request.
<code>getRemoteHost()</code>	Host name of the client machine that sent this request.
<code>getScheme()</code>	Returns the scheme used in the URL for this request (for example, <code>https</code> , <code>http</code> , <code>ftp</code> , etc.).
<code>getServerName()</code>	Name of the host server that received this request.
<code>getServerPort()</code>	Returns the port number used to receive this request.

The following Magercise shows you how to extract parameters from a service request.

4.1.18 Utility Classes

There are several utilities provided in the Servlet API. The first is the interface `javax.servlet.SingleThreadModel` that can make it easier to write simple servlets. If a servlet implements this marker interface, the hosting server knows that it should never call the

servlet's `service()` method while it is processing a request. That is, the server processes all service requests within a single thread. While this makes it easier to write a servlet, this can impede performance. A full discussion of this issue is located later in this course.

Two exception classes are included in the Servlet API. The exception `javax.servlet.ServletException` can be used when there is a general failure in the servlet. This notifies the hosting server that there is a problem. The exception `javax.servlet.UnavailableException` indicates that a servlet is unavailable. Servlets can report this exception at any time. There are two types of unavailability:

Permanent. The servlet is unable to function until an administrator takes some action. In this state, a servlet should write a log entry with a problem report and possible resolutions.

Temporary. The servlet encountered a (potentially) temporary problem, such as a full disk, failed server, etc. The problem can correct itself with time or may require operator intervention.

4.1.19 HTTP Support

Servlets that use the HTTP protocol are very common. It should not be a surprise that there is specific help for servlet developers who write them. Support for handling the HTTP protocol is provided in the package `javax.servlet.http`. Before looking at this package, take a look at the HTTP protocol itself. HTTP stands for the HyperText Transfer Protocol. It defines a protocol used by web browsers and servers to communicate with each other. The protocol defines a set of text-based request messages called HTTP methods. (Note:

The HTTP specification calls these HTTP methods; do not confuse this term with Java methods. Think of HTTP methods as messages requesting a certain type of response). The HTTP methods include:

- GET
- HEAD
- POST
- PUT
- DELETE
- TRACE
- CONNECT □ OPTIONS

For this course, you will only need to look at only three of these methods: GET, HEAD, and POST.

4.1.20 The HTTP GET Method

The HTTP GET method requests information from a web server. This information could be a file, output from a device on the server, or output from a program (such as a servlet or CGI script). An HTTP GET request takes the form:

GET URL <http version> Host: <target host> in addition to several other lines of information. For example, the following HTTP GET message is requesting the home page from the MageLang web site:

```
Host: GET / HTTP/1.1
      Connection: Keep-Alive    User-Agent: Mozilla/4.0 (compatible; MSIE 4.01;Windows NT)
www.magelang.com Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg
```

On most web servers, servlets are accessed via URLs that start with `/servlet/`. The following HTTP GET method is requesting the servlet `MyServlet` on the host www.magelang.com:

```
GET /servlet/MyServlet?name=Scott&
company=MageLang%20Institute HTTP/1.1 Connection: Keep-Alive
User-Agent: Mozilla/4.0 ( compatible; MSIE 4.01; Windows NT)
Host: www.magelang.com Accept:
image/gif, image/x-xbitmap, image/jpeg, image/pjpeg
```

The URL in this GET request invokes the servlet called `MyServlet` and contains two parameters, `name` and `company`. Each parameter is a name/value pair following the format `name=value`. The parameters are specified by following the servlet name with a question mark ('?'), with each parameter separated by an ampersand ('&').

Note the use of `%20` in the company's value. A space would signal the end of the URL in the GET request line, so it must be "URL encoded", or replaced with `%20` instead. As you will see later, servlet developers do not need to worry about this encoding as it will be automatically decoded by the `HttpServletRequest` class.

HTTP GET requests have an important limitation. Most web servers limit how much data can be passed as part of the URL name (usually a few hundred bytes.) If more data must be passed between the client and the server, the HTTP POST method should be used instead. It is important to note that the server's handling of a GET method is expected to be safe

and idempotent. This means that a GET method will not cause any side effects and that it can be executed repeatedly.

When a server replies to an HTTP GET request, it sends an HTTP response message back.

The header of an HTTP response looks like the following:

```
HTTP/1.1 200 Document follows Date: Tue, 14 Apr 1997 09:25:19 PST Server:  
JWS/1.1 Lastmodified: Mon, 17 Jun 1996 21:53:08 GMT Content-type: text/html  
Content-length: 4435
```

```
<4435 bytes worth of data -- the document body>
```

4.1.21 The HEAD Method

The HTTP HEAD method is very similar to the HTTP GET method. The request looks exactly the same as the GET request (except the word HEAD is used instead of GET), but the server only returns the header information.

HEAD is often used to check the following:

- The last-modified date of a document on the server for caching purposes
- The size of a document before downloading (so the browser can present progress information)
- The server type, allowing the client to customize requests for that server
- The type of the requested document, so the client can be sure it supports it

Note that HEAD, like GET, is expected to be safe and idempotent.

4.1.22 The POST Method

An HTTP POST request allows a client to send data to the server. This can be used for several purposes, such as

- Posting information to a newsgroup
- Adding entries to a web site's guest book
- Passing more information than a GET request allows

Pay special attention to the third bullet above. The HTTP GET request passes all its arguments as part of the URL. Many web servers have a limit to how much data they can accept as part of the URL. The POST method passes all of its parameter data in an input stream, removing this limit. A typical POST request might be as follows:

POST /servlet/MyServlet HTTP/1.1 User-Agent: Mozilla/4.0 (compatible; MSIE 4.01; Windows NT) Host: www.magelang.com Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, */ Content-type: application/x-www-form-urlencoded Content-length: 39

name=Scott&company=Magelang%20Institute

Note the blank line--this signals the end of the POST request header and the beginning of the extended information.

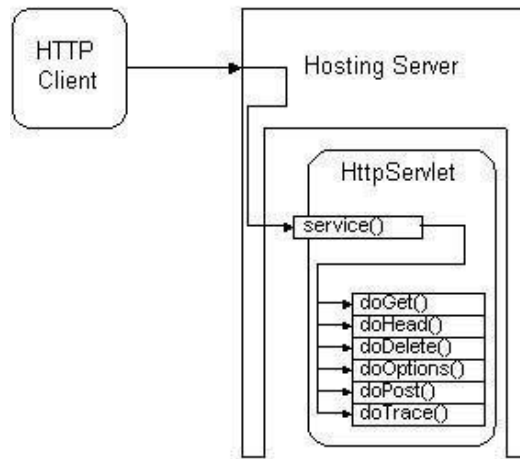
Unlike the GET method, POST is not expected to be safe nor idempotent; it can perform modifications to data, and it is not required to be repeatable.

4.1.23 HTTP Support Classes

Now that you have been introduced to the HTTP protocol, consider how the `javax.servlet.http` package helps you write HTTP servlets. The abstract class `javax.servlet.http.HttpServlet` provides an implementation of the `javax.servlet.Servlet` interface and includes a lot of helpful default functionality. The easiest way to write an HTTP servlet is to extend `HttpServlet` and add your own custom processing. The class `HttpServlet` provides an implementation of the `service()` method that dispatches the HTTP messages to one of several special methods. These methods are:

- `doGet()`
- `doHead()`
- `doDelete()`
- `doOptions()`
- `doPost()`
- `doTrace()` and correspond directly with the HTTP protocol methods.

As shown in the following diagram, the `service()` method interprets each HTTP method and determines if it is an HTTP GET, HTTP POST, HTTP HEAD, or other HTTP protocol method:



The class `HttpServlet` is actually rather intelligent. Not only does it dispatch HTTP requests, it detects which methods are overridden in a subclass and can report back to a client on the capabilities of the server. (Simply by overriding the `doGet()` method causes the class to respond to an HTTP `OPTIONS` method with information that `GET`, `HEAD`, `TRACE`, and `OPTIONS` are all supported. These capabilities are in fact all supported by the class's code). In another example of the support `HttpServlet` provides, if the `doGet()` method is overridden, there is an automatic response generated for the HTTP `HEAD` method. (Since the response to an HTTP `HEAD` method is identical to an HTTP `GET` method--minus the body of the message the `HttpServlet` class can generate an appropriate response to an HTTP `HEAD` request from the reply sent back from the `doGet()` method). As you might expect, if you need more precise control, you can always override the `doHead()` method and provide a custom response.

4.1.24 Using the HTTP Support Classes

When using the HTTP support classes, you generally create a new servlet that extends `HttpServlet` and overrides either `doGet()` or `doPost()`, or possibly both. Other methods can be overridden to get more finegrained control.

The HTTP processing methods are passed two parameters, an `HttpServletRequest` object and an `HttpServletResponse` object. The `HttpServletRequest` class has several convenience methods to help parse the request, or you can parse it yourself by simply reading the text of the request.

A servlet's `doGet()` method should

- Read request data, such as input parameters

- Set response headers (length, type, and encoding)
- Write the response data

It is important to note that the handling of a GET method is expected to be safe and idempotent. Handling is considered safe if it does not have any side effects for which users are held responsible, such as charging them for the access or storing data.

Handling is considered idempotent if it can safely be repeated. This allows a client to repeat a GET request without penalty. Think of it this way: GET should be "looking without touching." If you require processing that has side effects, you should use another HTTP method, such as POST. A servlet's doPost() method should be overridden when you need to process an HTML form posting or to handle a large amount of data being sent by a client. HTTP POST method handling is discussed in detail later. HEAD requests are processed by using the doGet() method of an HttpServlet. You could simply implement doGet() and be done with it; any document data that you write to the response output stream will not be returned to the client. A more efficient implementation, however, would check to see if the request was a GET or HEAD request, and if a HEAD request, not write the data to the response output stream.

Servlet support currently spans two packages:

javax.servlet: General Servlet Support	
Servlet	An interface that defines communication between a web server and a servlet. This interface defines the init(), service(), and destroy() methods (and a few others).
ServletConfig	An interface that describes the configuration parameters for a servlet. This is passed to the servlet when the web server calls its init() method. Note that the servlet should save the reference to the ServletConfig object, and define a getServletConfig() method to return it when asked. This interface defines how to get the initialization parameters for the servlet and the context under which the servlet is running.
ServletContext	An interface that describes how a servlet can get information about the server in which it is running. It can be retrieved via the getServletContext() method of the ServletConfig object.

ServletRequest	An interface that describes how to get information about a client request.
ServletResponse	An interface that describes how to pass information back to the client.
GenericServlet	A base servlet implementation. It takes care of saving the ServletConfig object reference, and provides several methods that delegate their functionality to the ServletConfig object. It also provides a dummy implementation for init() and destroy().
ServletInputStream	A subclass of InputStream used for reading the data part of a client's request. It adds a readLine() method for convenience.
ServletOutputStream	An OutputStream to which responses for the client are written.
ServletException	Should be thrown when a servlet problem is encountered.
UnavailableException	Should be thrown when the servlet is unavailable for some reason.

javax.servlet.http: Support for HTTP Servlets	
HttpServletRequest	A subclass of ServletRequest that defines several methods that parse HTTP request headers.
HttpServletResponse	A subclass of ServletResponse that provides access and interpretation of HTTP status codes and header information.
HttpServlet	A subclass of GenericServlet that provides automatic separation of HTTP request by method type. For example, an HTTP GET request will be processed by the service() method and passed to a doGet() method.
HttpUtils	A class that provides assistance for parsing HTTP GET and POST requests.

Servlet Examples

Now for an in-depth look at several servlets. These examples include:

- Generating Inline Content

- Processing HTTP Post Requests
- Using Cookies
- Maintaining Session Information
- Connecting to Databases

4.1.25 Generating Inline Content

Sometimes a web page needs only a small piece of information that is customized at runtime. The remainder of a page can be static information. To substitute only small amounts of information, some web servers support a concept known as server-side includes, or SSI.

If it supports SSI, the web server designates a special file extension (usually .shtml) which tells the server that it should look for SSI tags in the requested file. The JWS defines a special SSI tag called the `<servlet>` tag, for example:

```
<servlet code="DatePrintServlet">
    <param name=timezone value=pst>
</servlet>
```

This tag causes the invoking of a servlet named `DatePrintServlet` to generate some in-line content. SSI and servlets allow an HTML page designer to write a skeleton for a page, using servlets to fill in sections of it, rather than require the servlet to generate the entire page. This is very useful for features like pagehit counters and other small pieces of functionality. The `DatePrintServlet` servlet works just like a regular servlet except that it is designed to provide a very small response and not a complete HTML page. The output MIME type gets set to "text/plain" and not "text/html". Keep in mind that the syntax of server-side includes, if they are even supported, may vary greatly from one web server to another. In the following Magercise you create the `DatePrintServlet` and see how to use it in an HTML page.

4.1.26 Processing HTTP Post Requests

HTTP POST method processing differs from HTTP GET method processing in several ways. First, because POST is expected to modify data on the server, there can be a need to safely

handle updates coming from multiple clients at the same time. Second, because the size of the information stream sent by the client can be very large, the `doPost()` method must open an `InputStream` (or `Reader`) from the client to get any of the information. HTTP POST does not support sending parameters encoded inside of the URL as does the HTTP GET method. The problem of supporting simultaneous updates from multiple clients has been solved by database systems (DBMSs); unfortunately the HTTP protocol does not work well with database systems. This is because DBMSs need to maintain a persistent connection between a client and the DBMS to determine which client is trying to update the data. The HTTP protocol does not support this type of a connection as it is a message based, stateless protocol. Solving this problem is not easy and never elegant! Fortunately, the Servlet API defines a means to track client/server sessions. This is covered in the Maintaining Session Information section later in this course. Without session management tracking, one can resort to several different strategies. They all involve writing data to the client in hidden fields which is then sent back to the server. The simplest way to handle updates is to use an optimistic locking scheme based on date/time stamps. One can use a single date/time stamp for a whole form of data, or one could use separate date/time stamps for each "row" of information. Once the update strategy has been selected, capturing the data sent to the server via an HTTP POST method is straightforward. Information from the HTML form is sent as a series of parameters (name/value pairs) in the `InputStream` object. The `HttpUtils` class contains a method `parsePostData()` that accepts the raw `InputStream` from the client and return a `Hashtable` with the parameter information already processed. A really nice feature is that if a parameter of a given name has multiple values (such is the case for a column name with multiple rows), then this information can be retrieved from the `Hashtable` as an array of type `String`. In the following

Magercise, you will be given skeleton code that implements a pair of servlets that display data in a browser as an editable HTML form. The structure of the data is kept separate from the actual data. This makes it easy to modify this code to run against arbitrary tables from a JDBC-connected database.

4.2 Cookies

For those unfamiliar with cookies, a cookie is a named piece of data maintained by a browser, normally for session management. Since HTTP connections are stateless, you can use a cookie to store persistent information accross multiple HTTP connections. The Cookie

class is where all the "magic" is done. The HttpSession class, described next, is actually easier to use. However, it doesn't support retaining the information across multiple browser sessions.

To save cookie information you need to create a Cookie, set the content type of the HttpServletResponse response, add the cookie to the response, and then send the output. You must add the cookie after setting the content type, but before sending the output, as the cookie is sent back as part of the HTTP response header.

```
private static final String SUM_KEY
= "sum";    ... int sum = ...; //
get old value and add to it
Cookie theCookie = new Cookie (SUM_KEY,
Integer.toString(sum));
response.setContentType("text/html");
response.addCookie(theCookie);
```

It is necessary to remember that all cookie data are strings. You must convert information like int data to a String object. By default, the cookie lives for the life of the browser session. To enable a cookie to live longer, you must call the setMaxAge(interval) method.

When positive, this allows you to set the number of seconds a cookie exists. A negative setting is the default and destroys the cookie when the browser exits. A zero setting immediately deletes the cookie. Retrieving cookie data is a little awkward. You cannot ask for the cookie with a specific key. You must ask for all cookies and find the specific one you are interested in. And, it is possible that multiple cookies could have the same name, so just finding the first setting is not always sufficient. The following code finds the setting of a single-valued cookie:

```
int sum = 0;
Cookie theCookie = null;
Cookie cookies[] =
request.getCookies();    if (cookies
!= null)
{
    for(int i=0,
n=cookies.length; i < n; i++)
    {
        theCookie = cookies[i];
        if (theCookie.getName().equals(SUM_KEY))
        {
            try
            {
                sum =
Integer.parseInt(theCookie.getValue());
```

```

        } catch
(NumberFormatException ignored)
    {
        sum = 0;
    }
    break;
    }
    }
}

```

4.2.1 Maintaining Session Information

A session is a continuous connection from the same browser over a fixed period of time. (This time is usually configurable from the web server. For the JWS, the default is 30 minutes.) Through the implicit use of browser cookies, HTTP servlets allow you to maintain session information with the `HttpSession` class. The `HttpServletRequest` provides the current session with the `getSession(boolean)` method. If the boolean parameter is true, a new session will be created when a new session is detected. This is, normally, the desired behavior. In the event the parameter is false, then the method returns null if a new session is detected.

```

public void doGet (HttpServletRequest
request,           HttpServletResponse response)
throws ServletException, IOException {
    HttpSession session =
request.getSession(true);
    //

```

...

Once you have access to an `HttpSession`, you can maintain a collection of key-value-paired information, for storing any sort of session-specific data. You automatically have access to the creation time of the session with `getCreationTime()` and the last accessed time with `getLastAccessedTime()`, which describes the time the last servlet request was sent for this session.

To store session-specific information, you use the `putValue(key, value)` method. To retrieve the information, you ask the session with `getValue(key)`. The following example demonstrates this, by continually summing up the integer value of the `Addend` parameter. In the event the value is not an integer, the number of errors are also counted.

```

private static final String SUM_KEY =
"session.sum"; private static final String

```

```

ERROR_KEY = "session.errors";    Integer sum =
(Integer) session.getValue(SUM_KEY);    int ival =
0;    if (sum != null)
    {
        ival =
sum.intValue();    }
try
    {
        String addendString =
request.getParameter("Addend");    int addend
=
Integer.parseInt (addendString);    sum = new
Integer(ival + addend);    session.putValue
(SUM_KEY, sum);    }    catch
(NumberFormatException e)
    {
        Integer errorCount =
(Integer) session.getValue(ERROR_KEY);    if
(errorCount == null)
        {
            errorCount = new
Integer(1);

        }
        el
        se
        {
            errorCount = new
Integer(errorCount.intValue(
)+1);    }
        session.putValue (ERROR_KEY,
errorCount);
    }
}

```

As with all servlets, once you've performed the necessary operations, you need to generate some output. If you are using sessions, it is necessary to request the session with `HttpServletRequest.getSession()` before generating any output.

```

response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("<html>" +
    "<head><title>Session
Information</title></head>" +
    "<body bgcolor=\"#FFFFFF\">" +
    "<h1>Session Information</h1><table>");
out.println ("<tr><td>Identifier</td>");
    out.println ("<td>" + session.getId() +
"</td></tr>");    out.println
("<tr><td>Created</td>");

```

```

out.println("<td>" + new Date(
session.getCreationTime()) + "</td></tr>");
out.println("<tr><td>Last Accessed</td>");
out.println("<td>" + new Date(
session.getLastAccessedTime()) + "</td></tr>");
out.println("<tr><td>New Session?</td>");
out.println("<td>" + session.isNew() +
"</td></tr>");    String names[] =
session.getValueNames();    for (int i=0,
n=names.length; i<n; i++) {        out.println
("<tr><td>" + names[i] + "</td>");        out.println
("<td>" + session.getValue (names[i])
+ "</td></tr>");
    }
    out.println("</table></center></body></html>");
out.close();

```

The complete code example shown above is available for testing. One thing not demonstrated in the example is the ability to end a session, where the next call to `request.getSession(true)` returns a different session. This is done with a call to `invalidate()`. In the event a user has browser cookies disabled, you can encode the session ID within the `HttpServletResponse` by calling its `encodeUrl()` method.

4.3 Connecting to Database

It is very common to have servlets connect to databases through JDBC. This allows you to better control access to the database by only permitting the middle-tier to communicate with the database. If your database server includes sufficient simultaneous connection licenses, you can even setup database connections once, when the servlet is initialized, and pool the connections between all the different service requests.

The following demonstrates sharing a single `Connection` between all service requests.

To find out how many simultaneous connections the driver supports, you can ask its `DatabaseMetaData` and then create a pool of `Connection` objects to share between service requests. In the `init()` method connect to the database.

```

Connection con = null;

```

```

public void init (ServletConfig cfg) throws
ServletException
{
    super.init
    (cfg);    //
    Load
    driver
        String name = cfg.getInitParameter("driver");
        Class.forName(name);    // Get Connection    con
    = DriverManager.getConnection (urlString); }

```

In the **doGet()** method retrieve database information.

```

public    void    doGet    (HttpServletRequest    request,
HttpServletRequest response)    throws
ServletException, IOException {
    response.setContentType("text/html");    //
    Have    browser    ignore    cache    -    force
        reload    response.setHeader ("Expires", "Mon, 01
Jan 1990 00:00:00 GMT");
    Statement stmt = null;
    ResultSet result = null;
    try
    {
        // Submit query
        stmt = con.createStatement();        result =
stmt.executeQuery ("SELECT programmer, cups " +
"FROM JoltData ORDER BY cups    DESC;");        //
    Create output        PrintWriter out =
response.getWriter();        while(result.next())
    {
        // Generate output from ResultSet
    } }
    finally
    {
        if (result != null)
        {
            result.close
            e();
        }
        if (stmt !=
        null)
        {
            stmt.close
            e();
        } }
    out.flush
    ()

```

```

;
out.close()
;
}

```

In the **destroy()** method disconnect from the database.

```

public void destroy()
{
    super.destroy();
    con.close();
}

```

It is not good practice to leave a database connection permanently open, so this servlet should not be installed as a permanent servlet. Having it as a temporary servlet that closes itself down after a predefined period of inactivity allows the sharing of the database connection with requests that coincide, reducing the cost of each request. You can also save some information in the HttpSession to possible page through the result set.

4.3.1 Security Issues

As with Java applets, Java servlets have security issues to worry about, too.

a) The Servlet Sandbox

A servlet can originate from several sources. A webmaster may have written it; a user may have written it; it may have been bought as part of a third-party package or downloaded from another web site. Based on the source of the servlet, a certain level of trust should be associated with that servlet. Some web servers provide a means to associate different levels of trust with different servlets. This concept is similar to how web browsers control applets, and is known as "sandboxing".

A servlet sandbox is an area where servlets are given restricted authority on the server. They may not have access to the file system or network, or they may have been granted a more trusted status. It is up to the web server administrator to decide which servlets are granted this status. Note that a fully trusted servlet has full access to the server's file system and networking capabilities. It could even perform a System.exit(), stopping the web server...

b) Access Control Lists (ACLs)

Many web servers allow you to restrict access to certain web pages and servlets via access control lists (ACLs). An ACL is a list of users who are allowed to perform a specific function in the server. The list specifies:

- What kind of access is allowed
- What object the access applies to
- Which users are granted access

Each web server has its own means of specifying an ACL, but in general, a list of users is registered on the server, and those user names are used in an ACL. Some servers also allow you to add users to logical groups, so you can grant access to a group of users without specifying all of them explicitly in the ACL. ACLs are extremely important, as some servlets can present or modify sensitive data and should be tightly controlled, while others only present public knowledge and do not need to be controlled.

c) Threading Issues

A web server can call a servlet's `service()` method for several requests at once. This brings up the issue of thread safety in servlets. But first consider what you do not need to worry about: a servlet's `init()` method. The `init()` method will only be called once for the duration of the time that a servlet is loaded. The web server calls `init()` when loading, and will not call it again unless the servlet has been unloaded and reloaded.

In addition, the `service()` method or `destroy()` method will not be called until the `init()` method has completed its processing.

Things get more interesting when you consider the `service()` method. The `service()` method can be called by the web server for multiple clients at the same time. (With the JSDK 2.0, you can tag a servlet with the `SingleThreadModel` interface. This results in each call to `service()` being handled serially. Shared resources, such as files and databases, can still have concurrency issues to handle.)

If your `service()` method uses outside resources, such as instance data from the servlet object, files, or databases, you need to carefully examine what might happen if multiple calls are made to `service()` at the same time. For example, suppose you had defined a counter in your servlet class that keeps track of how many `service()` method invocations are currently running:

```
private int counter = 0;
```

Next, suppose that your `service()` method contained the following code:

```
int myNumber = counter + 1; // line
1 counter = myNumber;      //
line 2 // rest of the code in the
service() method counter = counter -
1;
```

What would happen if two service() methods were running at the same time, and both executed line 1 before either executed line 2? Both would have the same value for myNumber, and the counter would not be properly updated.

For this situation, the answer might be to synchronize the access to the counter variable:

```
synchronized(this)
{
    myNumber = counter + 1;
    counter = myNumber;
}
// rest of code in the service() method
synchronized(this)
{
    counter =
counter - 1 ;
}
```

This ensures that the counter access code is executed only one thread at a time. There are several issues that can arise with multi-threaded execution, such as deadlocks and coordinated interactions.

4.4 Learner Activity

1. What are Servlet? How are they different from Applets?
2. How does a web server handle HTTP request?
3. Describe GET and POST methods.
4. Describe the inner working of a servlet engine.
5. Describe various parts of a basic servlet.

4.5 Coding Activity

Students will learn how to create and deploy a simple Java Servlet that responds with a personalized greeting based on user input.

References

1. Deitel, H. M., & Deitel, P. J. (2021). *Java How to Program, Early Objects* (11th ed.). Pearson Education.
2. Flanagan, D., & Evans, B. (2020). *Java in a Nutshell* (7th ed.). O'Reilly Media.
3. Oaks, S. (2020). *Java Performance: The Definitive Guide*. O'Reilly Media.
4. Eckel, B. (2023). *Thinking in Java* (5th ed.). Pearson.
5. Sierra, K., & Bates, B. (2023). *Head First Java* (2nd ed.). O'Reilly Media.
6. Goetz, B., & Peierls, T. (2022). *Java Concurrency in Practice*. Addison-Wesley Professional.
7. Sedgewick, R., & Wayne, K. (2020). *Introduction to Programming in Java: An Interdisciplinary Approach* (2nd ed.). Addison-Wesley Professional.
8. Kamath, U., & Choppella, K. (2021). *Mastering Java Machine Learning*. Packt Publishing.
9. Needham, T. C. (2022). *Learn Java: A Crash Course Guide to Learn Java in 1 Week*. Independently Published.
10. Long, J., & Bastani, K. (2020). *Cloud Native Java*. O'Reilly Media.
11. Burns, A., & Wellings, A. (2021). *Concurrent and Real-Time Programming in Java*. Springer.
12. Paul, J. (2020). *Grokking the Java Interview*. Independently Published.
13. Liang, Y. D. (2020). *Introduction to Java Programming and Data Structures*. Pearson Education.
14. Bloch, J. (2020). *Effective Java* (3rd ed.). Addison-Wesley Professional.
15. Oracle. (2021). "Java Magazine: Top Articles." Retrieved from [blogs.oracle.com](https://blogs.oracle.com/java-magazine).

The IT qualification at Richfield College stands as a beacon of academic innovation and professional readiness. It equips students with the skills and credentials necessary for thriving in the IT industry. By combining foundational knowledge, practical expertise, and global recognition, the program not only prepares students for immediate employment but also sets them on a trajectory for long-term career success.

